



Deep Learning, Machine Learning und Künstliche Intelligenz – SS 26

Gelesen von Prof. Dr. Daniel Gaida

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können ein Deep Learning Projekt selbstständig durchführen, indem sie

- für eine gegebene Anwendung ein geeignetes künstliches neuronales Netz auswählen können,
- dieses in keras implementieren und trainieren
- und bei schlechten Validierungsergebnissen Lösungsstrategien zur Verbesserung der Ergebnisse anwenden können,

um später eigene Deep Learning Projekte zur Bilderkennung, Zeitreihenanalyse, Textanalyse und -generierung, ... umsetzen zu können.

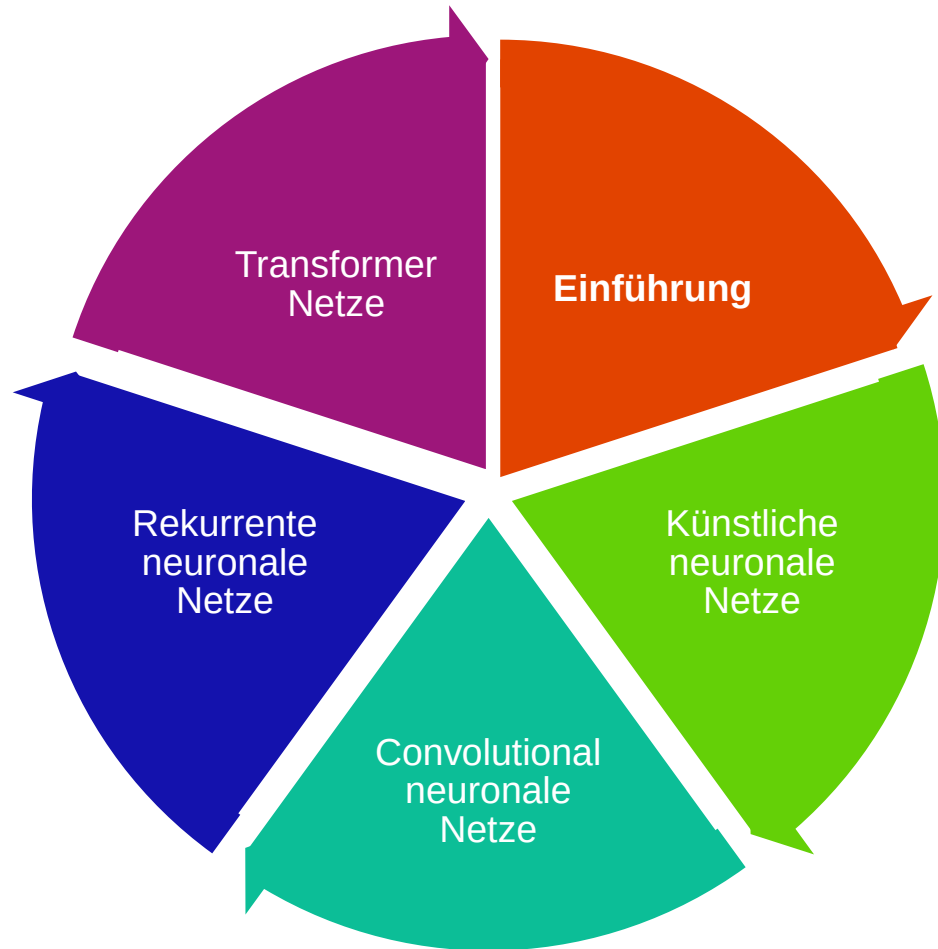
Überprüfung des Lernziels: S. fortlaufende Übung in diesem Lernraum II.

Lernraum II: Deep Learning

- Künstliche neuronale Netze
 - Grundlagen
 - Training
 - Implementierung
 - Lösungsstrategien, um gute Ergebnisse zu erzielen
- Convolutional neuronale Netze
 - Bilderkennung
- Rekurrente neuronale Netze
 - Zeitreihen, Text
- Transformer Netze
 - Text, Bilderkennung
- Anwendungen
 - Entwickeln eines Chatbots
 - Objektdetektion und -segmentierung

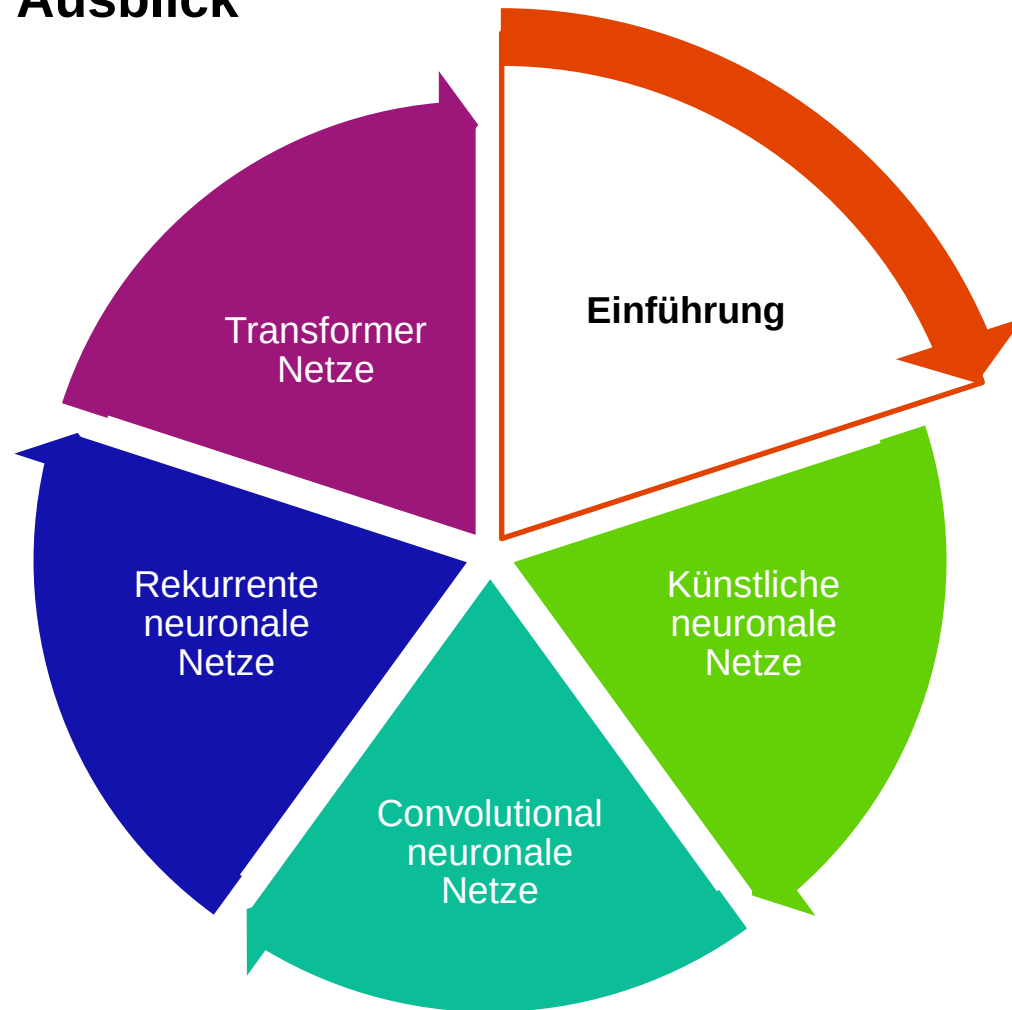
Deep Learning

Ausblick



Deep Learning

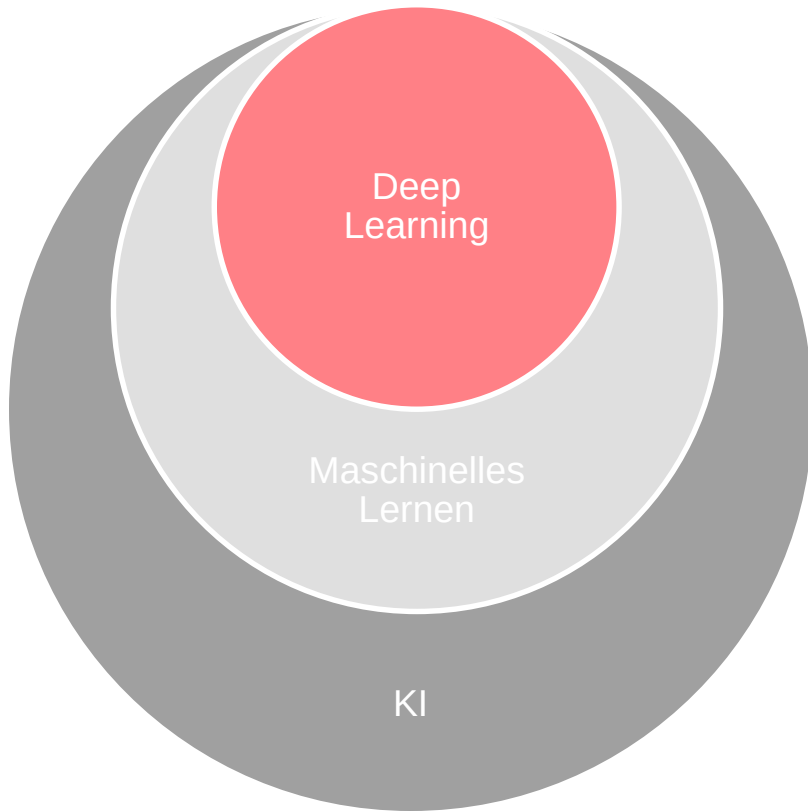
Ausblick



Einführung

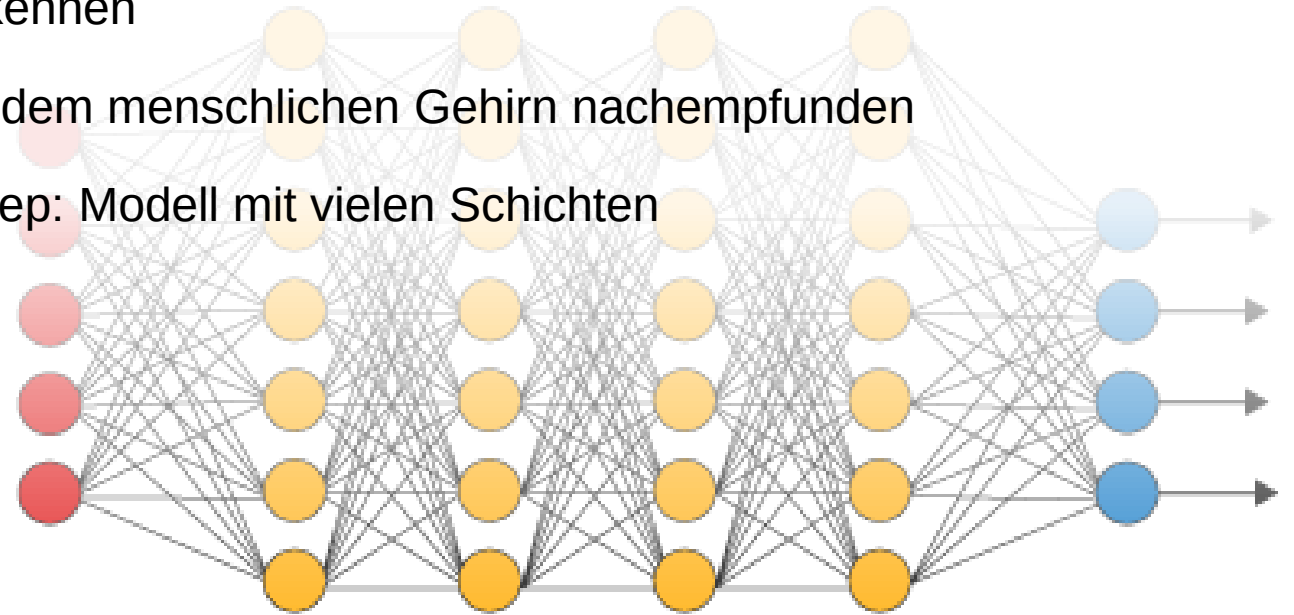
- Begriffserklärung
- Deep Learning
- Warum Deep Learning?

KI – Maschinelles Lernen – Deep Learning

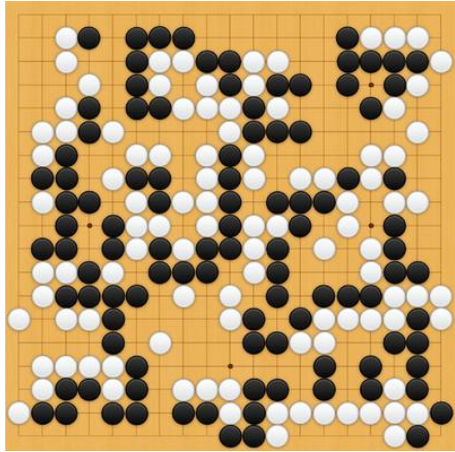


Deep Learning

- Verwendet neuronale Netze, um Muster in Daten zu erkennen
- Ist dem menschlichen Gehirn nachempfunden
- Deep: Modell mit vielen Schichten



Brettspiele Go



<https://www.top500.org/news/google-latest-alphago-ai-program-crushes-its-predecessor/>

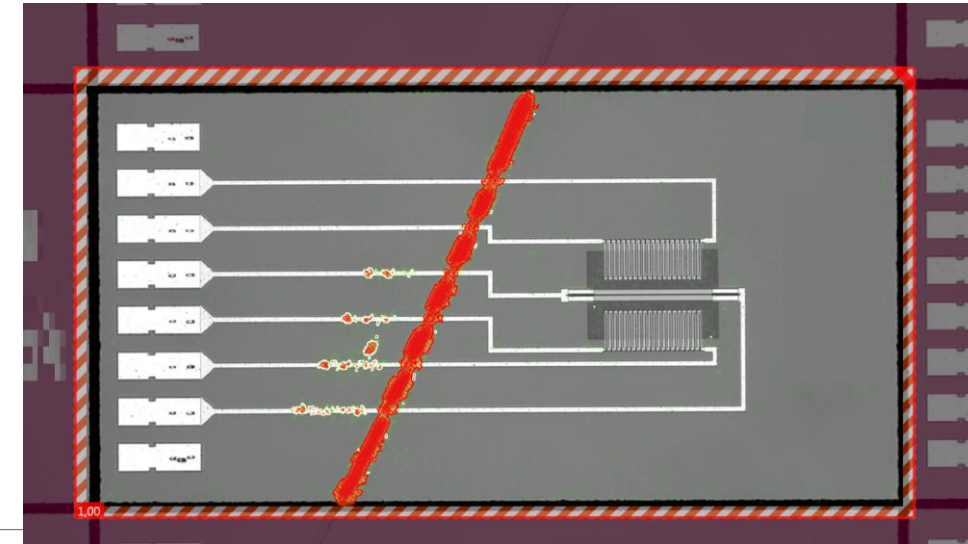
Spracherkennung



Übersetzen und Generieren von Texten



Automatische Optische Inspektion



Quelle: Patrick S., Hahn-Schickard

Bilderkennung und -generierung



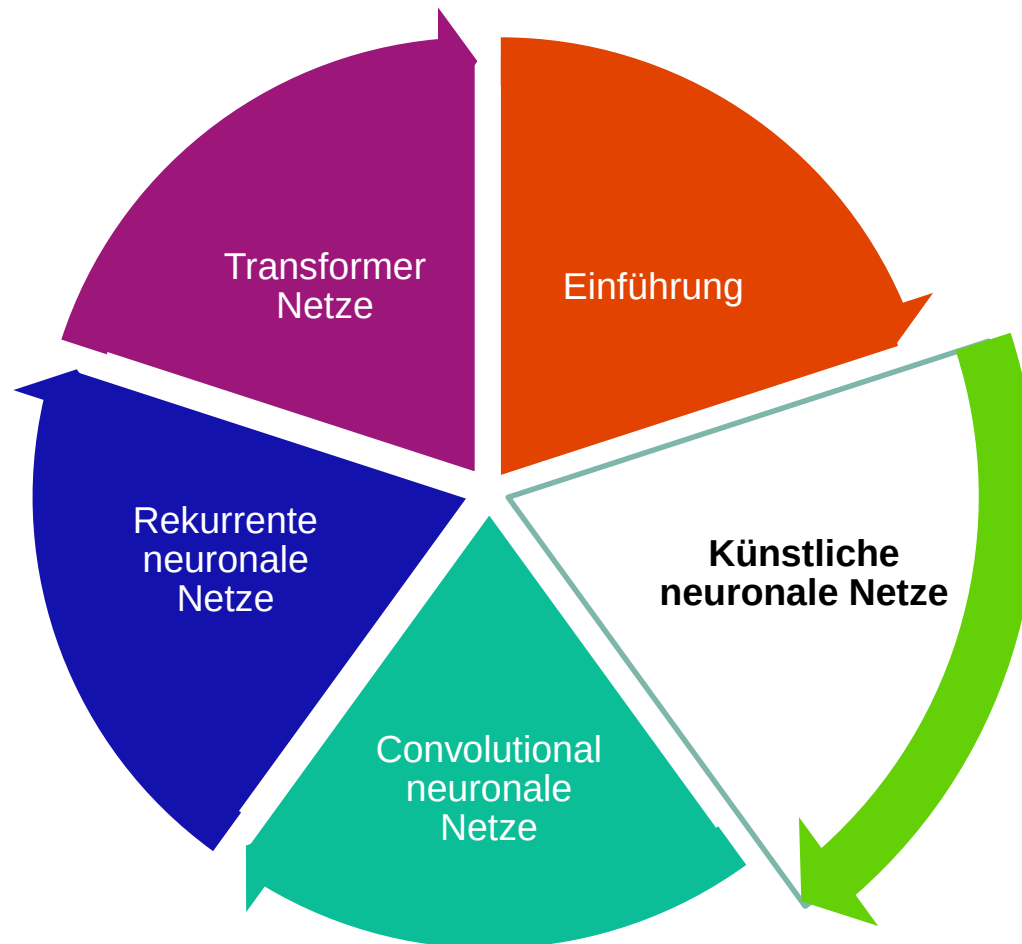
01.05.2026

Seite 7

Technology
Arts Sciences
TH Köln

Deep Learning

Ausblick

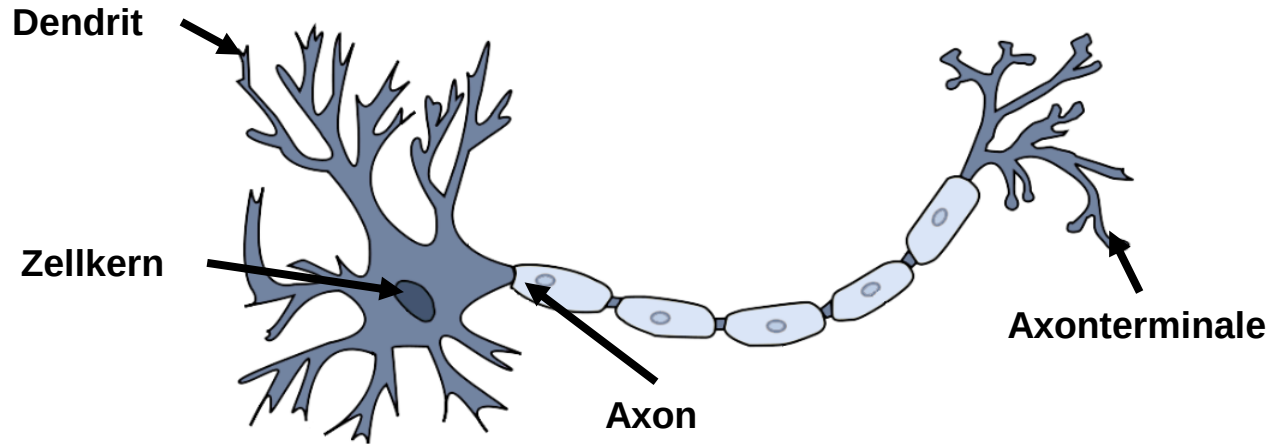


Künstliche neuronale Netze: Part 1

- Einführung
 - Das biologische Neuron
 - Das künstliche Neuron
 - Das XOR Problem
- Training neuronaler Netze

Künstliche Neuronale Netze - KNN

Das künstliche Neuron

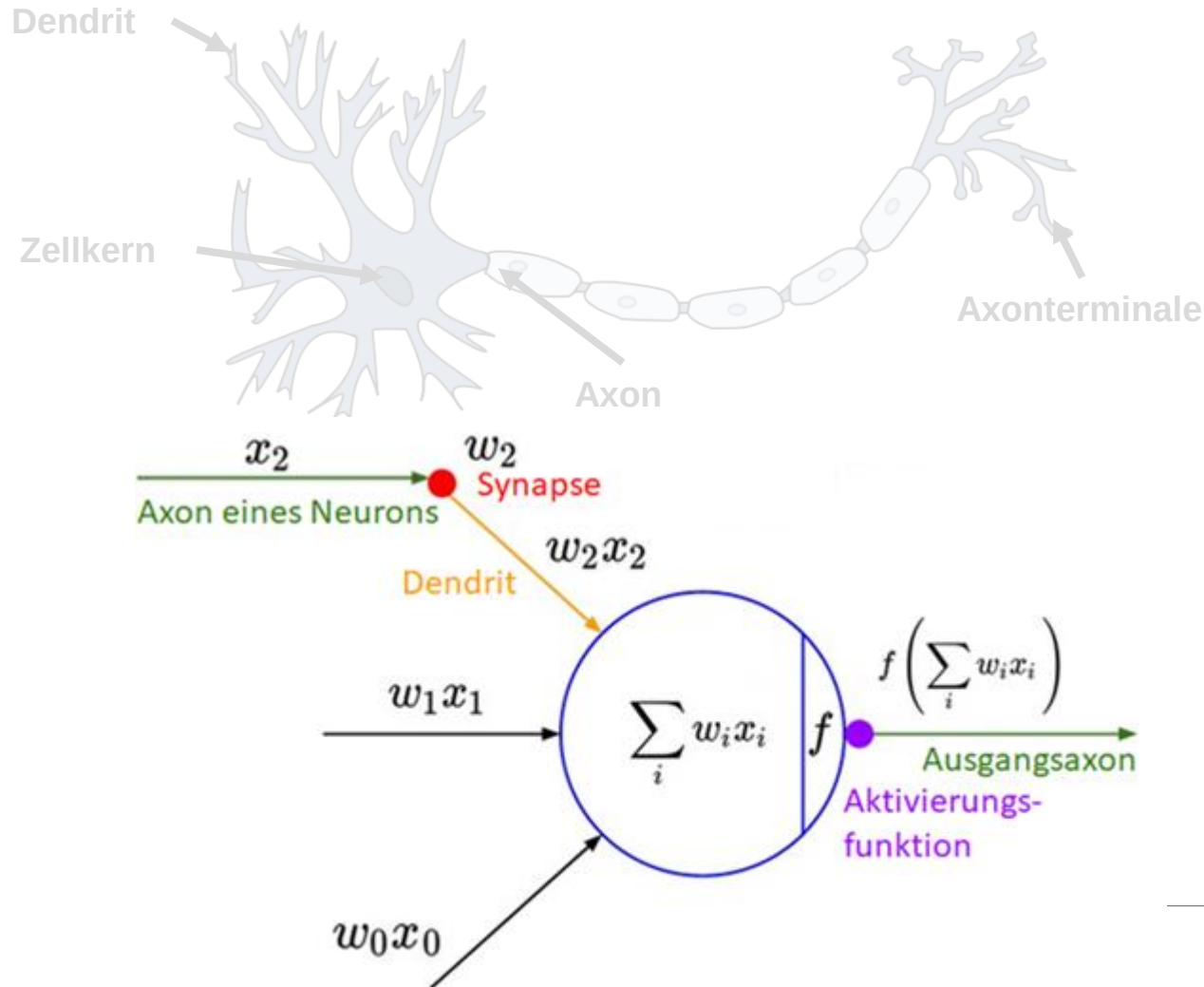


Das biologische Neuron

- Elektrisch erregbare Zelle (Aufnahme, Verarbeitung und Weiterleitung von Informationen)
- Grundeinheit eines biologischen Gehirns
- Überträgt elektrische Signale von den Dendriten entlang des Axons zu den Terminalen
- Signale werden von Neuron zu Neuron übergeben
- Ausgangssignal entsteht nur, wenn die Eingabe ein Schwellwert überschreitet:
 - Das Neuron feuert
- Auf diese Weise nehmen wir Licht, Töne, Druck, Wärme, usw. wahr

Künstliche Neuronale Netze - KNN

Das künstliche Neuron

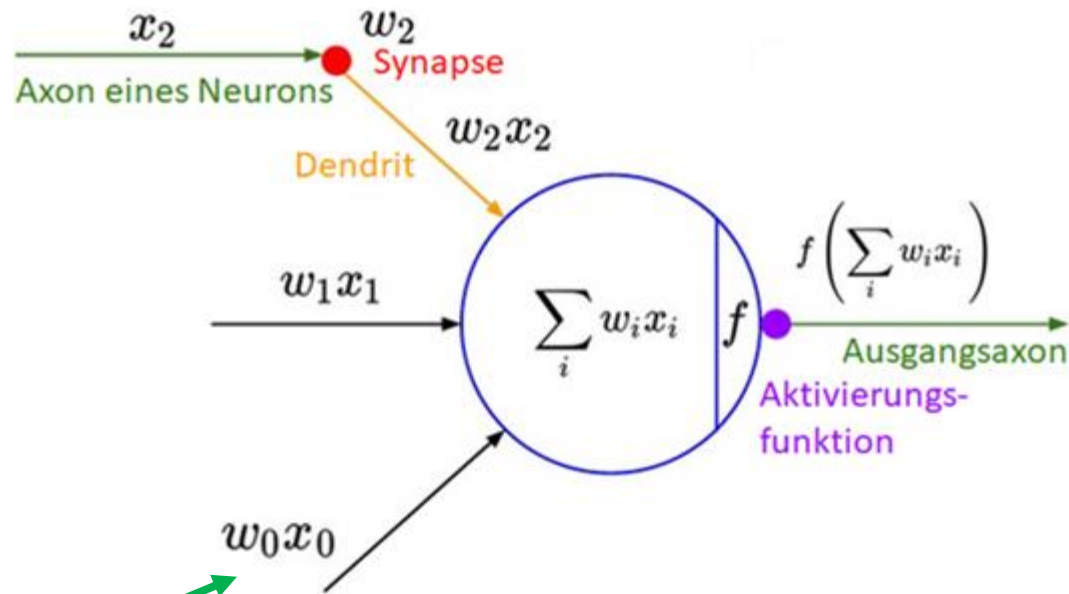


Das künstliche Neuron - Perzeptron

- Grundeinheit eines neuronalen Netzes
- Ein- und Ausgänge sind Zahlen (anstatt binäre Ein/Aus Werte)
- Jede Eingangsverbindung ist mit einem Gewicht verbunden
- Berechnung der gewichteten Summe aller Eingänge
- Anwenden einer Aktivierungsfunktion
- Verknüpfung dieser Bausteine bilden künstliche Neuronale Netze

Künstliche Neuronale Netze - KNN

Das künstliche Neuron



Aktivierungsfunktion
(Stufenfunktion, Signumfunktion):

$$f(z) = f(\boldsymbol{\omega}^T \cdot \mathbf{x}) = \begin{cases} 0 & \text{falls } \boldsymbol{\omega}^T \cdot \mathbf{x} < 0 \\ 1 & \text{sonst} \end{cases}$$

Vorwärtspropagation

- Gewichtete Summe:

$$z = \sum_{i=0}^n \omega_i \cdot x_i = \boldsymbol{\omega}^T \cdot \mathbf{x}$$

mit $\boldsymbol{\omega} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix}$ $\mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$

Bias ist im
Gewichtsvektor
enthalten

w_0 wird als Bias
bezeichnet. x_0 ist
kein Eingang des
Neurons

Künstliche Neuronale Netze - KNN

Das künstliche Neuron – Bias

Bias ω_0 :

- Analogie: y-Achsenabschnitt (hier: n)

$$\omega = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix} \quad \mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

Bias ist im Gewichtsvektor enthalten

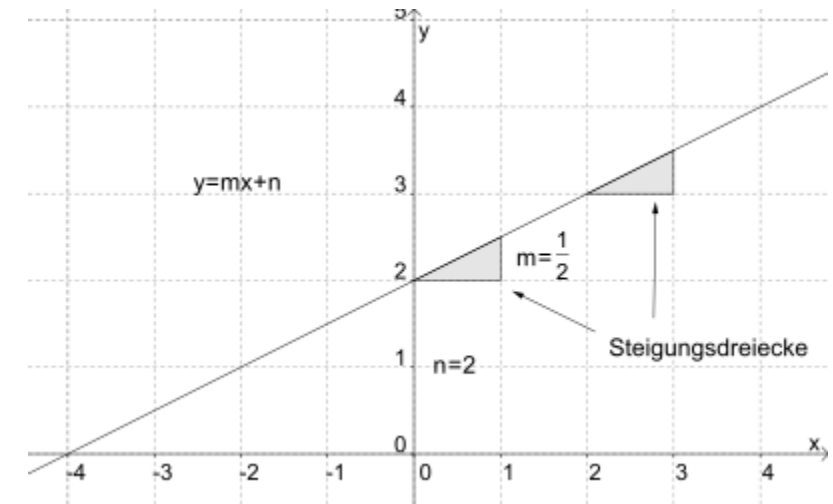


Bild: https://de.wikipedia.org/wiki/Lineare_Funktion

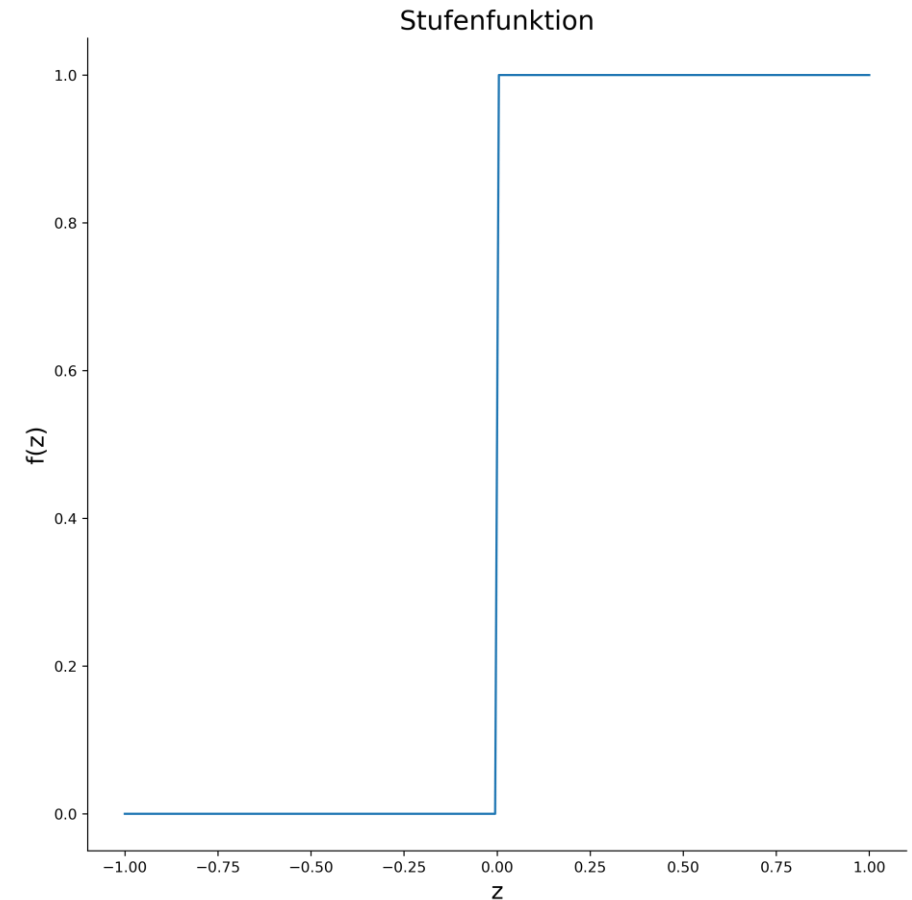
Künstliche Neuronale Netze - KNN

Das künstliche Neuron – Stufenfunktion

Aktivierungsfunktion

- Stufenfunktion, Signumfunktion:

$$f(z) = f(\omega^T \cdot x) = \begin{cases} 0 & \text{falls } \omega^T \cdot x < 0 \\ 1 & \text{sonst} \end{cases}$$



Künstliche Neuronale Netze - KNN

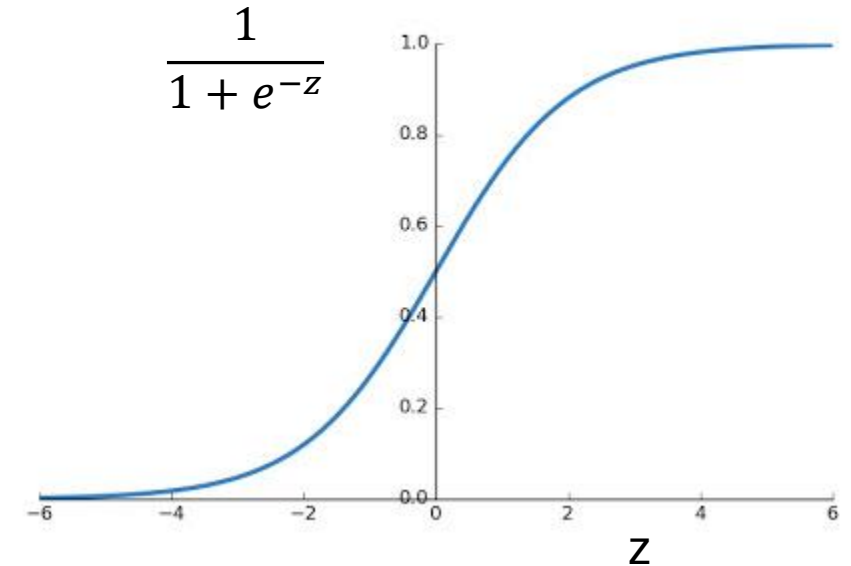
Das künstliche Neuron – Sigmoid-Funktion

Aktivierungsfunktion

- Sigmoid-Funktion (logistisches Wachstum):

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Vorteil: differenzierbar

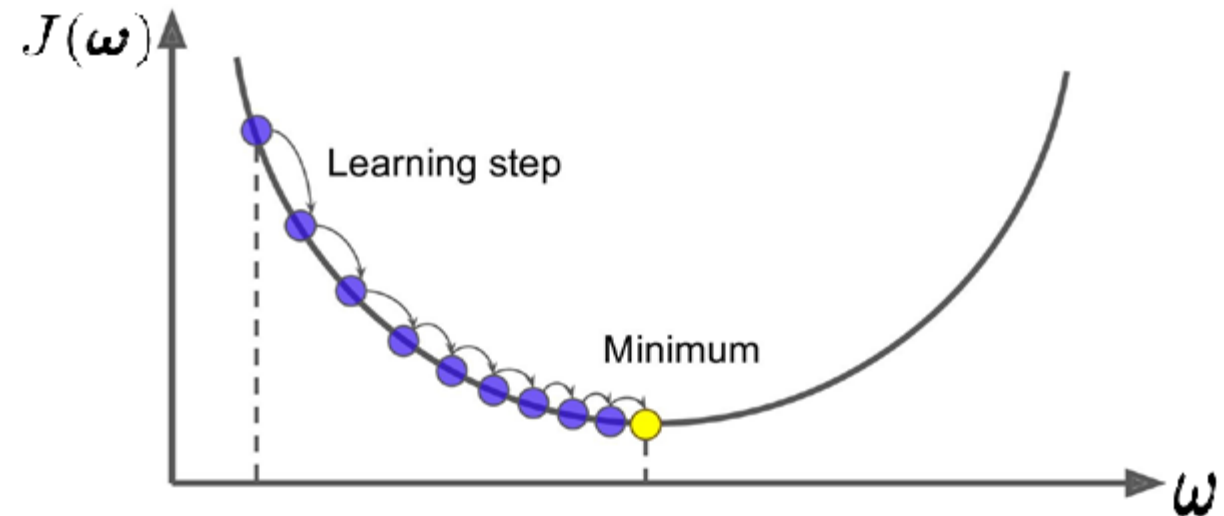


Künstliche Neuronale Netze - KNN

Das künstliche Neuron – Gradientenabstieg

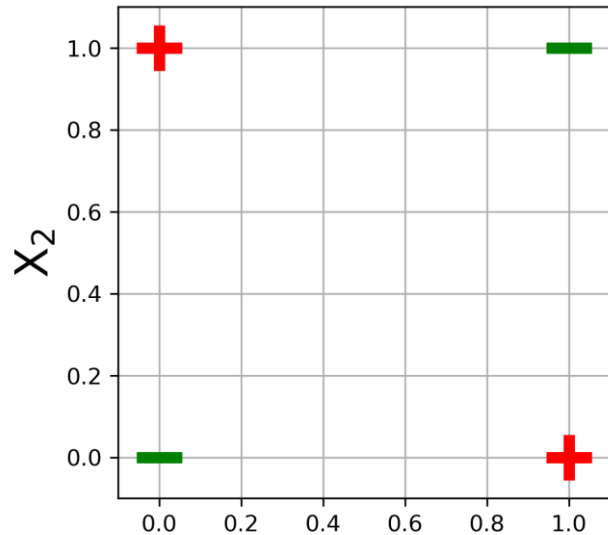
Gradientenabstieg

- Iterativer Algorithmus, der angibt ob Gewichte kleiner oder größer gemacht werden müssen, um bessere Vorhersagen zu erzielen.
- Gewichte werden so verändert, dass man in Richtung des steilsten Abstiegs der Kostenfunktion geht



Künstliche Neuronale Netze - KNN

Das XOR-Problem

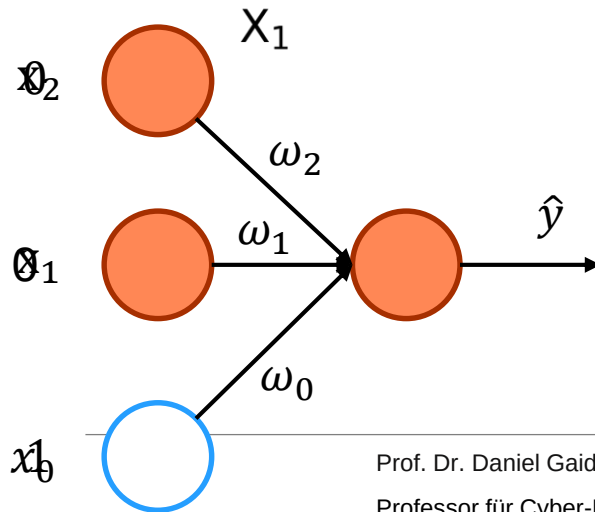


Das XOR-Problem

- Beispiel für ein nicht-linear separierbares Problem

Daten erstellen

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad W = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \end{bmatrix}$$



Eingabeschicht

Neuronen in der Eingabeschicht leiten Eingangsdaten weiter

- 2 Neuronen in der Eingabeschicht, da Eingangsdaten 2-dimensional
- 3. Dimension ist nur der Bias

Prof. Dr. Daniel Gaida

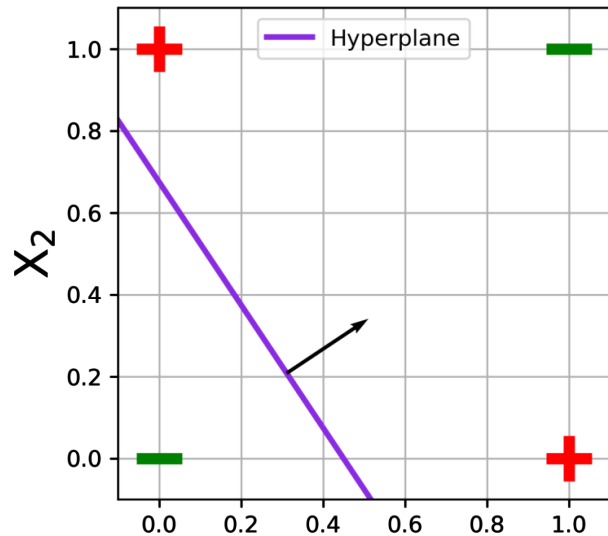
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Patrick S., Hahn-Schickard

Künstliche Neuronale Netze - KNN

Das XOR-Problem



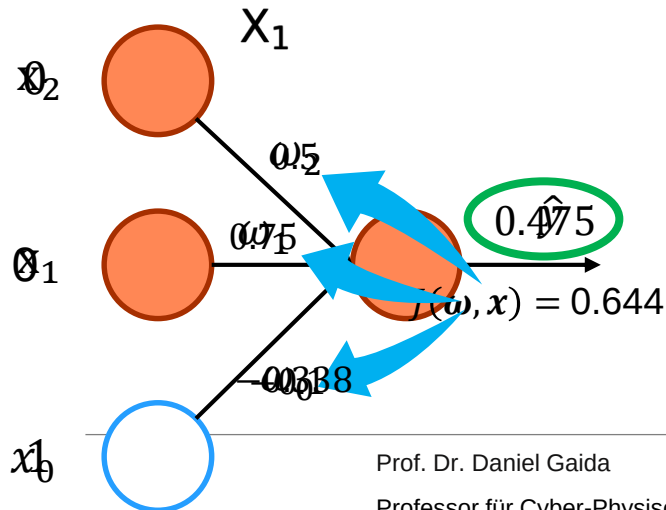
Das XOR-Problem

- Beispiel für ein nicht-linear separierbares Problem

Daten erstellen

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad W = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \end{bmatrix}$$

2. Entwicklung (Aktivierungsfunktion) Step, $\eta = \frac{1}{2}$



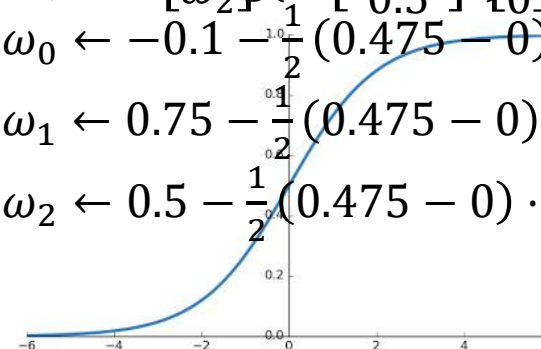
$$J(\omega, x) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

$$z = \omega_0 + \omega_1 x_1 + \omega_2 x_2 = -0.1 + 0.75 \cdot 1 + 0.5 \cdot 0 = 0.475$$

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases} = 0$$

$$J(\omega, x) = -[0 \log(0) + 1 \log(1-0)] = -\log(1) = 0.644$$

- $\omega_0 \leftarrow -0.1 - \frac{1}{2} (0.475 - 0) \cdot 1 = -0.338$
- $\omega_1 \leftarrow 0.75 - \frac{1}{2} (0.475 - 0) \cdot 0 = 0.75$
- $\omega_2 \leftarrow 0.5 - \frac{1}{2} (0.475 - 0) \cdot 0 = 0.5$



Quelle: Patrick S., Hahn-Schickard

Künstliche Neuronale Netze - KNN

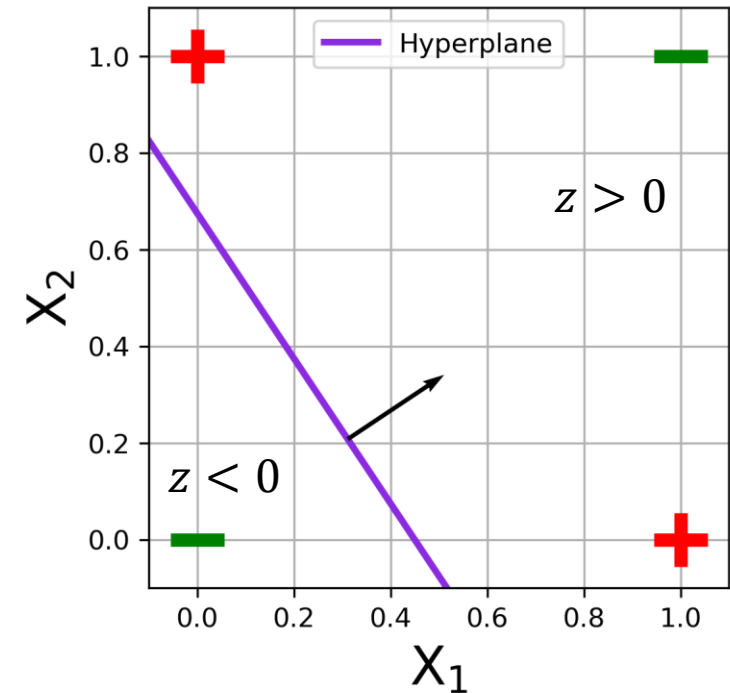
Warum ist der Gewichtsvektor eine Hyperebene?

Ein Neuron führt eine lineare Transformation durch:

- Berechnet: $z = \omega_1 x_1 + \omega_2 x_2 + \omega_0$
- Gewichte (ω_1, ω_2) bestimmen die Neigungsrichtung.
- Bias (ω_0) verschiebt die Grenze.

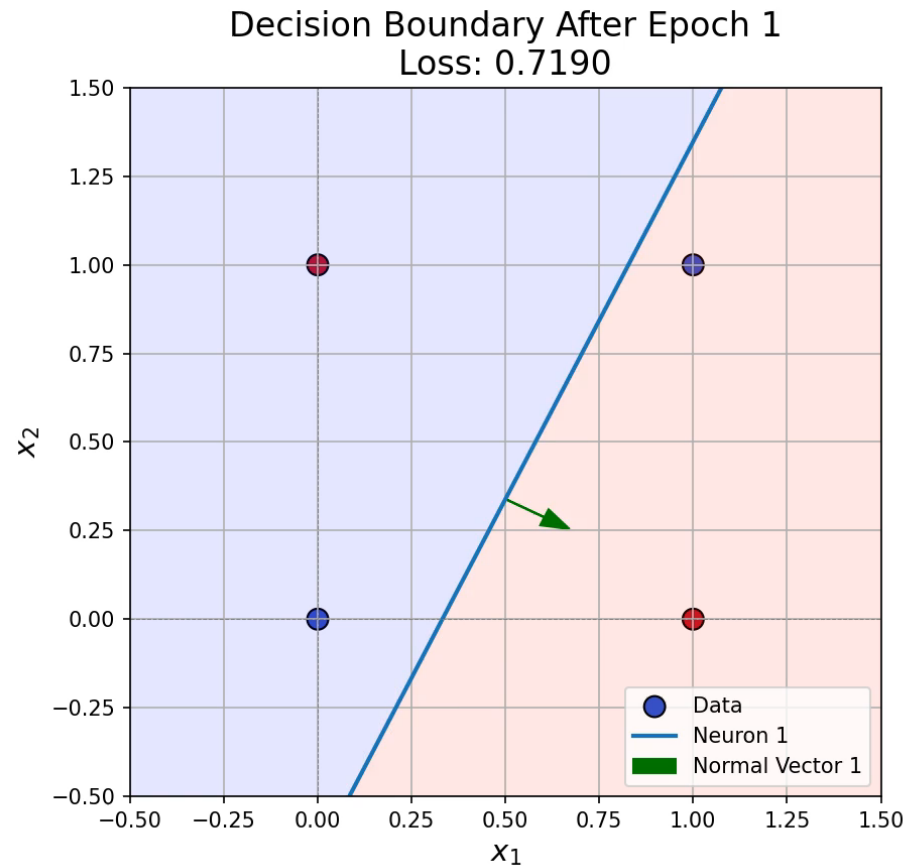
Entscheidungsgrenze = Nullstelle der Funktion

- Setze $z = 0 \rightarrow z = \omega_1 x_1 + \omega_2 x_2 + \omega_0 = 0$
- Ergibt eine Gerade in 2D, eine Ebene in 3D, eine Hyperebene in höheren Dimensionen.

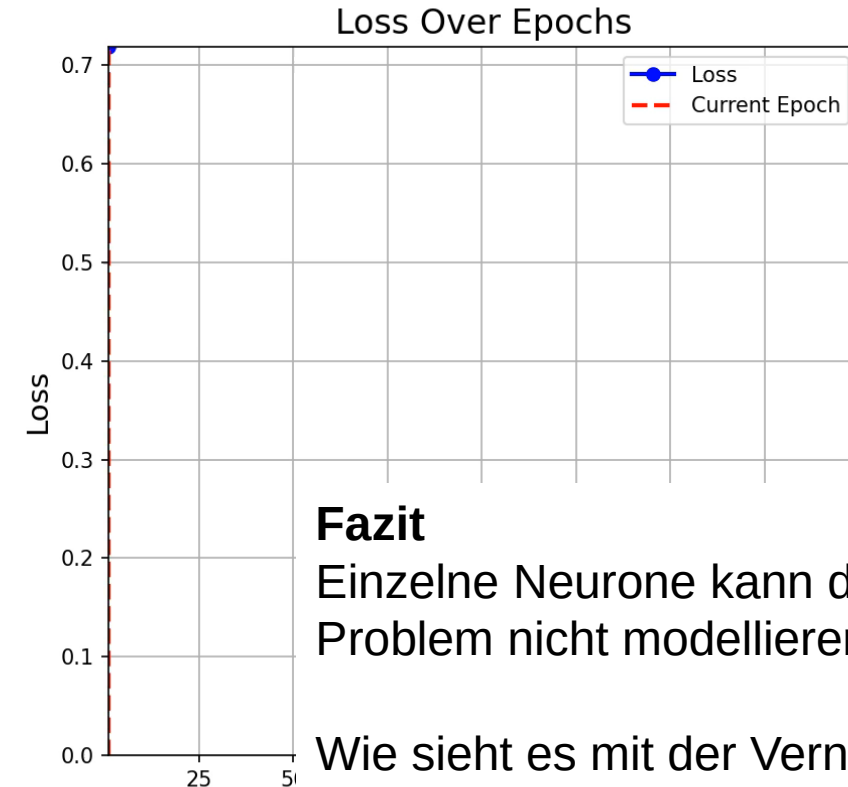


Künstliche Neuronale Netze - KNN

Das XOR-Problem – Ein Neuron



Kostenfunktion



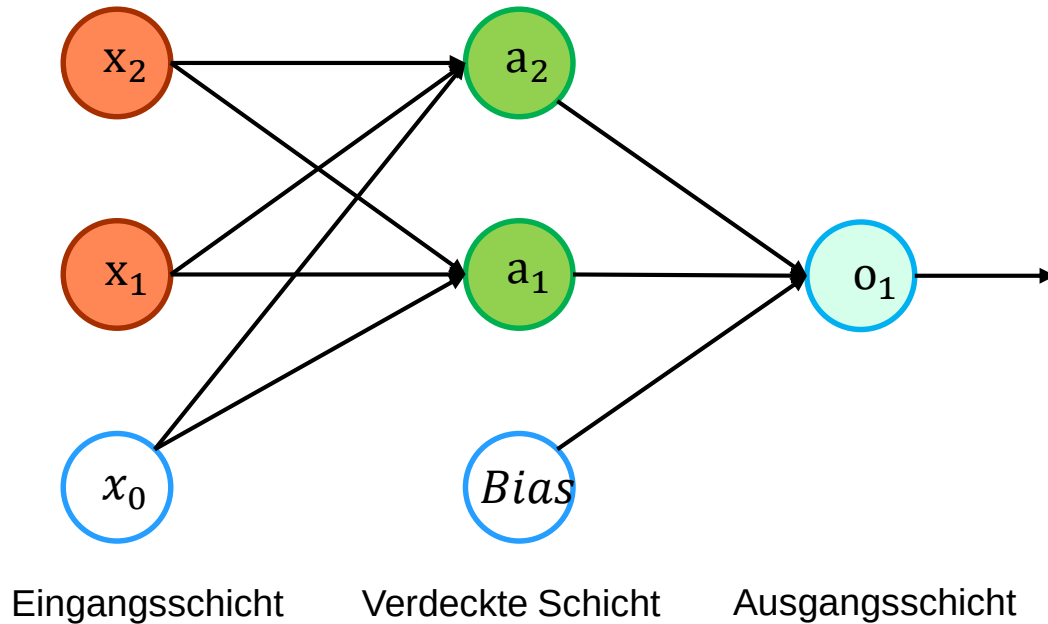
Fazit

Einzelne Neurone kann das XOR-Problem nicht modellieren

Wie sieht es mit der Vernetzung mehrerer künstlicher Neuronen aus?

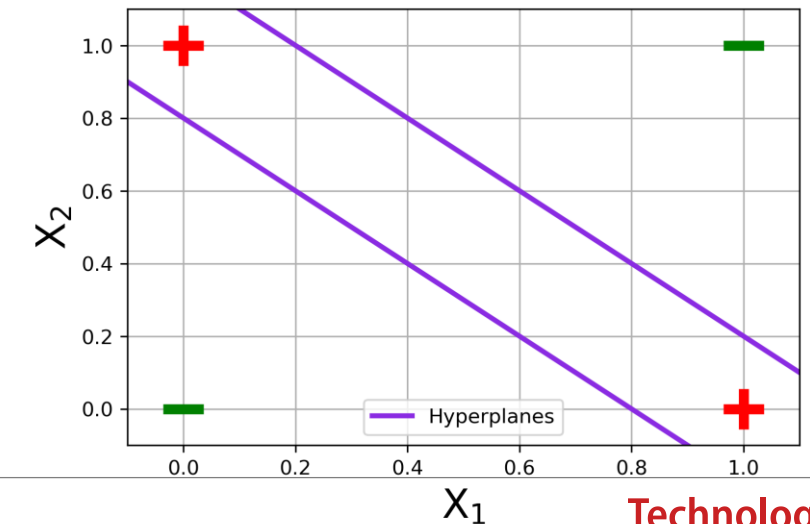
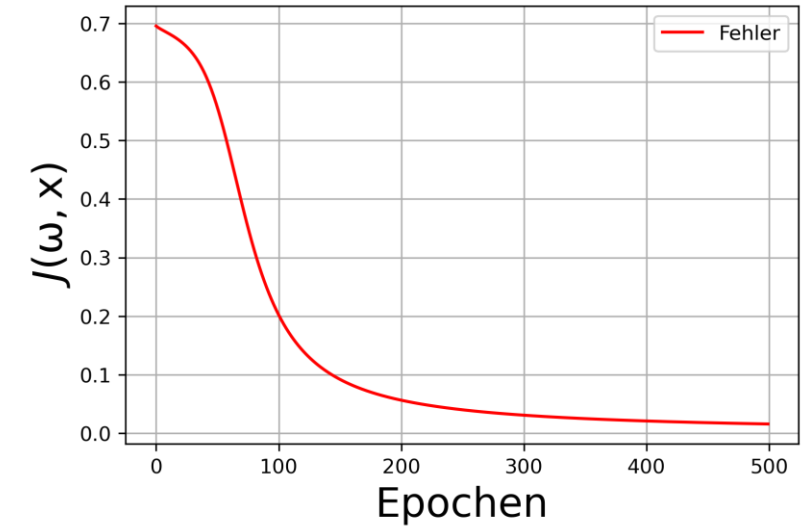
Künstliche Neuronale Netze - KNN

Das XOR-Problem



Drei Neuronen: a_1 , a_2 , o_1

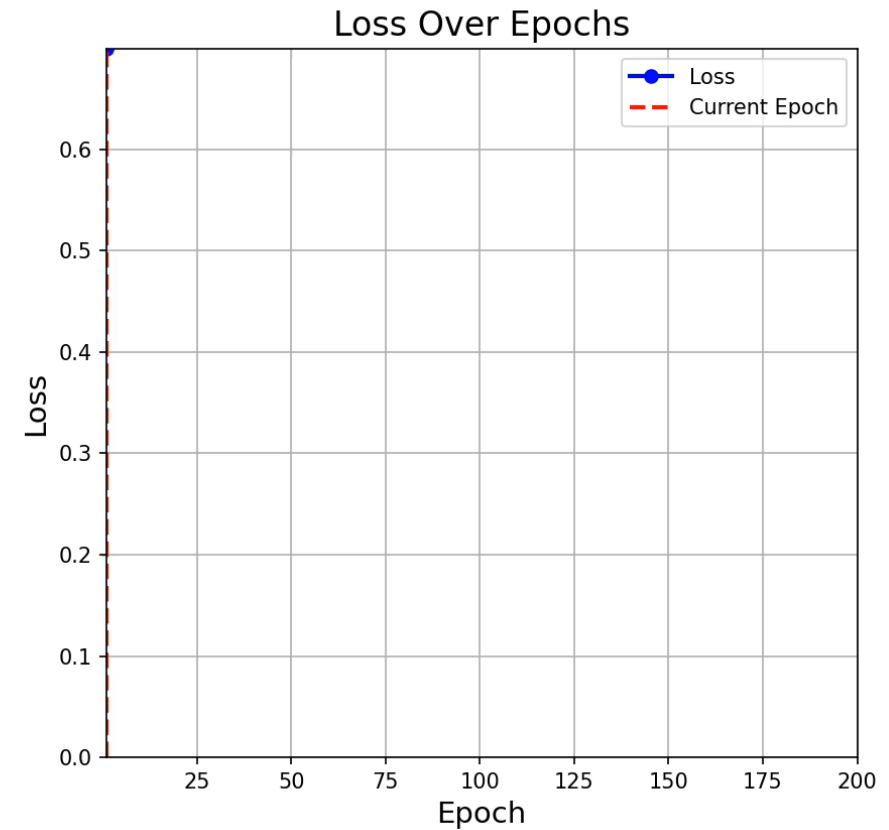
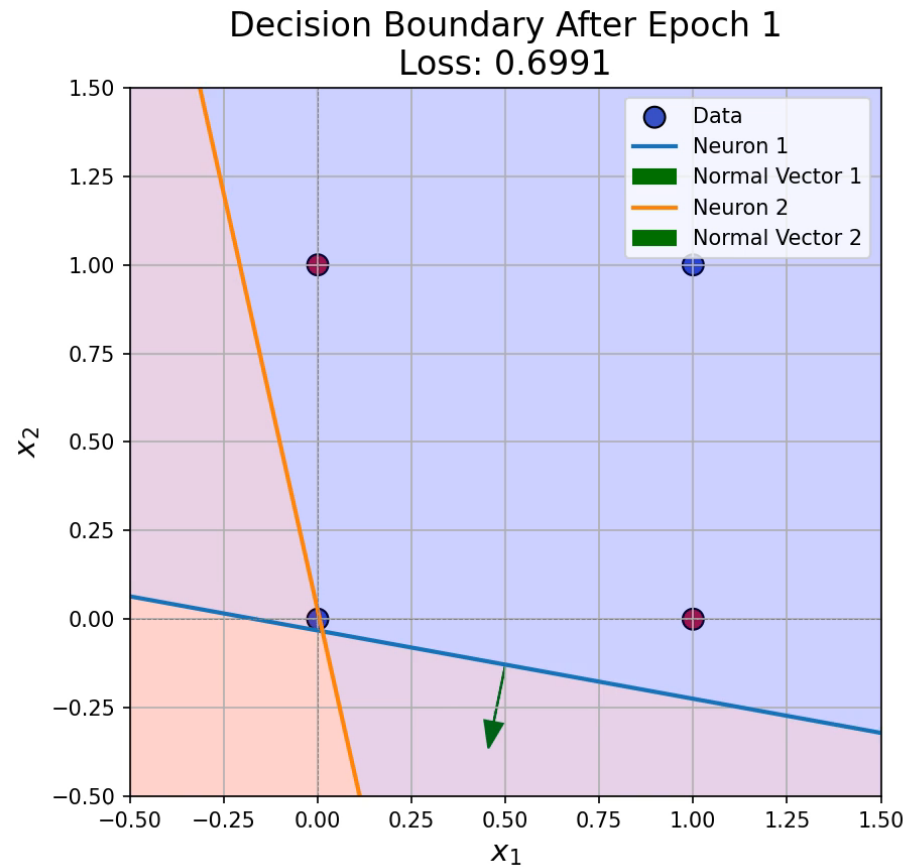
Erste Schicht (a_1 , a_2) definiert zwei Hyperebenen



Aktivierung: sigmoid, $\eta = 0.5$

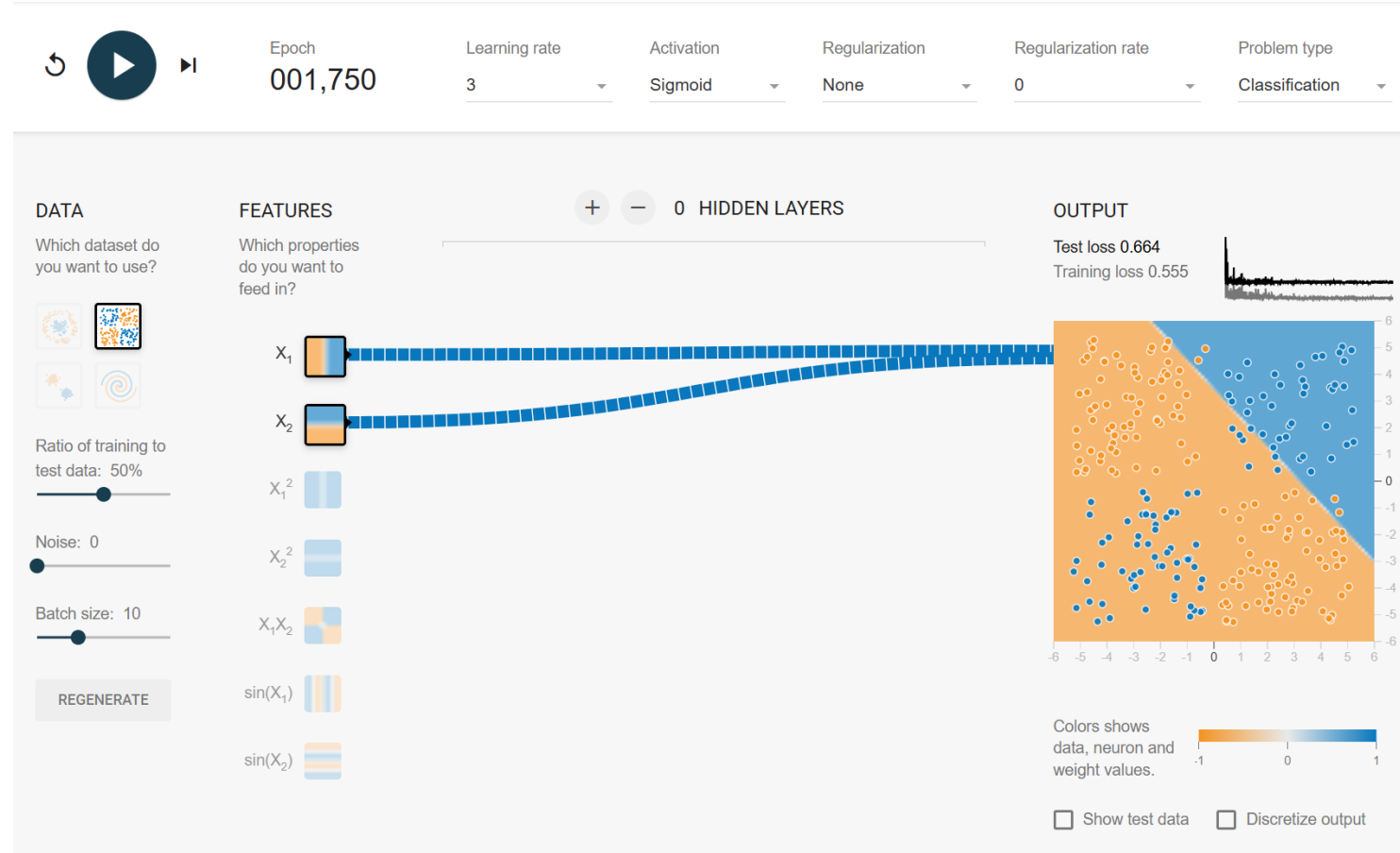
Künstliche Neuronale Netze - KNN

Das XOR-Problem



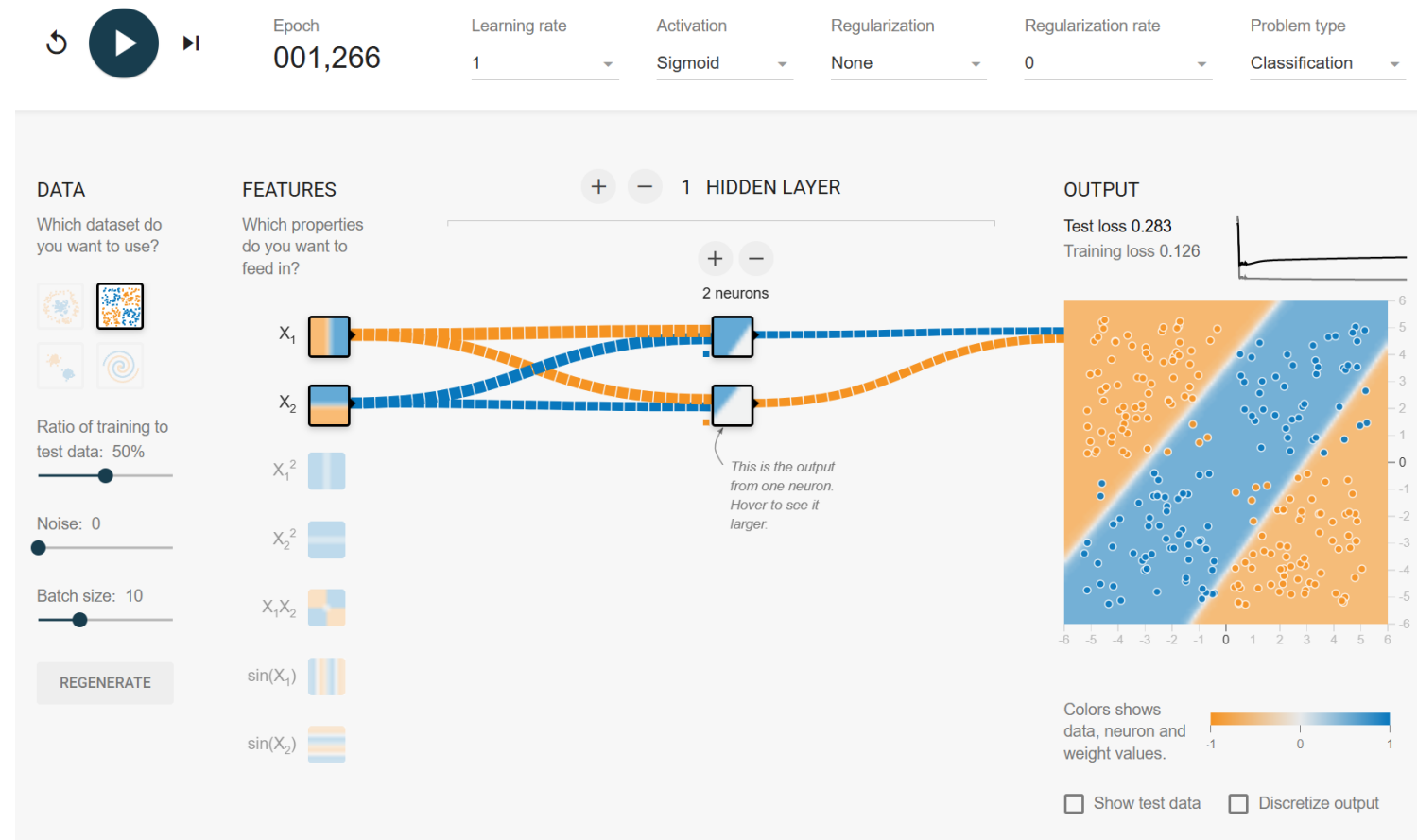
Künstliche Neuronale Netze - KNN

Übung: Experimentieren Sie mit Google Playground: <https://playground.tensorflow.org/>



Künstliche Neuronale Netze - KNN

Übung: Experimentieren Sie mit Google Playground: <https://playground.tensorflow.org/>



Künstliche Neuronale Netze - KNN

Das XOR-Problem – Training eines neuronalen Netzes

Training eines neuronalen Netzes:

1. Gewichtsinitialisierung

Für jeden Datenpunkt:

2. Forward Propagation
3. Aktivierung
4. Kostenfunktion (berechnen des Prognosefehlers)
5. Gewichtsaktualisierung (Gradient Descent Step)

Viele Begriffe eingeführt:

Forward Propagation, Aktivierungsfunktion, Kostenfunktion, Gradient Descent

Künstliche Neuronale Netze - KNN

Fragen I

Das Perzeptron (ein künstliches Neuron) feuert immer dann, wenn

- Alle Eingänge größer gleich 0 sind
- Falls mindestens ein Eingang kleiner 0 ist
- Summe der gewichteten Eingabe größer gleich 0 ist

Ein Perzeptron mit zwei Eingängen löst folgende Probleme

- Linear separierbares Problem in einer zweidimensionalen Ebene
- Linear separierbares Problem in einer dreidimensionalen Ebene
- Nicht-linear separierbares Problem in einer zweidimensionalen Ebene (XOR-Problem)
- Keines der genannten Probleme

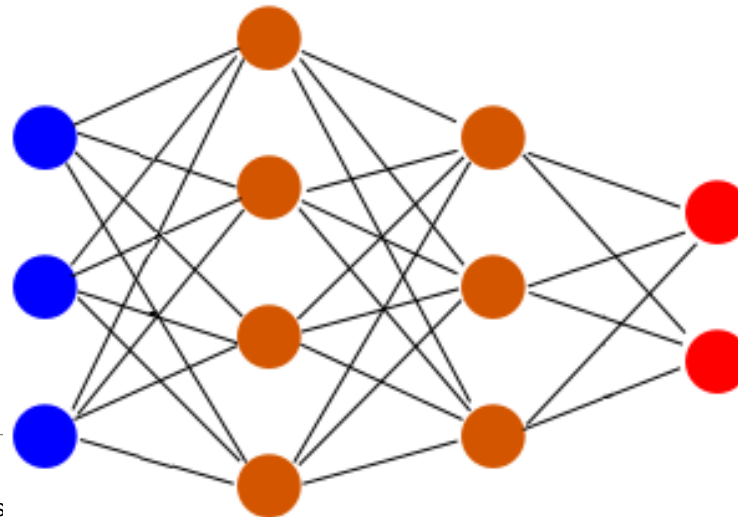
Künstliche Neuronale Netze - KNN

Problemstellung herkömmlicher Computer

Problem	Computer	Mensch
Tausende große Zahlen schnell multiplizieren	Leicht	Schwer
Gesichter auf einem Foto einer Menschenmenge suchen	Schwer	Leicht

→ KI: neue Algorithmen finden, die auf neuartige Weise versuchen, derartige Probleme zu lösen

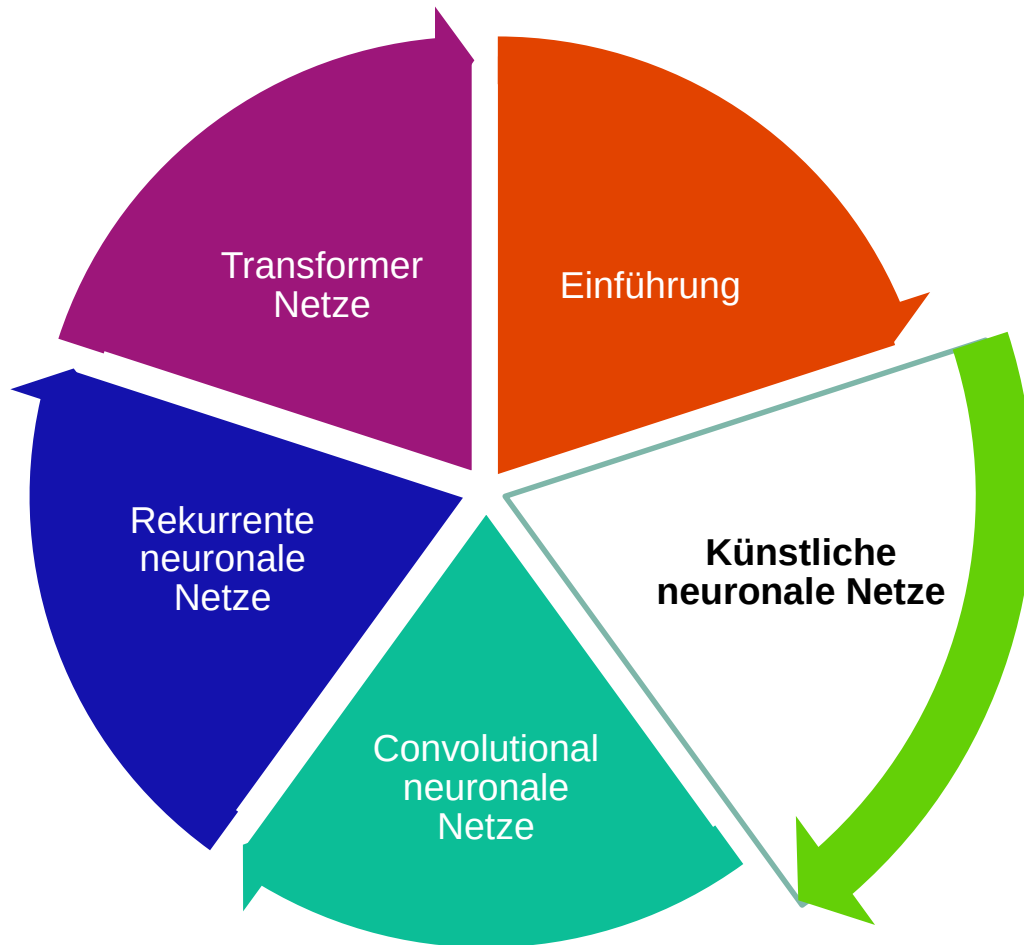
Künstliches neuronales Netz



- Übernimmt eine Eingabe und liefert eine Ausgabe
- Inspiriert durch das menschliche Gehirn
- Kann Muster erkennen
- Muss trainiert werden
- KNNs: Herzstück des Deep Learnings

Deep Learning

Ausblick



Künstliche neuronale Netze: Part 1

- Einführung
- Training neuronaler Netze
 - Optimierungsalgorithmus
 - Aktivierungsfunktionen
 - Kostenfunktion
 - Performancemetriken

Deep Learning - Zutaten

Modellarchitektur

- Anzahl Schichten
- Aktivierungsfunktion

Kostenfunktion

Definiert was ein gutes neuronales Netz ist

Optimierungsalgorithmus

Minimiert die Kostenfunktion, indem es Gewichte des NNs variiert

Training des NNs

Minimierung der Kostenfunktion

Neuronale Netze programmieren in keras

Das XOR-Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

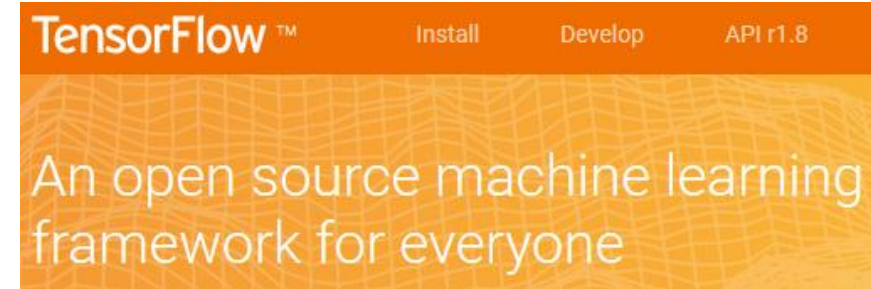
Daten importieren

```
model = Sequential()
model.add(Dense(units=2, activation='sigmoid', input_shape=(2,)))
model.add(Dense(units=1, activation='sigmoid'))
```

```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=500, verbose=0)
```

```
classes = (model.predict(x_test) > 0.5).astype("int32")
print("Predictions:", classes)
```



Keras: The Python Deep Learning library



Neuronale Netze programmieren in keras

Das XOR-Beispiel

```
import numpy as np
```

```
# Dummy-Daten (XOR-Problem)
def load_data():
    x_train = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    y_train = np.array([[0], [1], [1], [0]])
    return (x_train, y_train), (x_train, y_train)

(x_train, y_train), (x_test, y_test) = load_data()
```

Daten importieren

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Hier absoluter Sonderfall, dass Training = Testdaten sind.

Neuronale Netze programmieren in keras

Das XOR-Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

```
model = Sequential()
model.add(Dense(units=2, activation='sigmoid', input_shape=(2,)))
model.add(Dense(units=1, activation='sigmoid'))
```

```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```

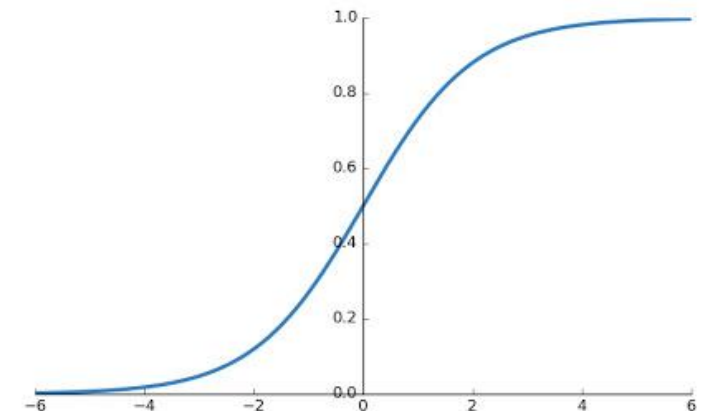
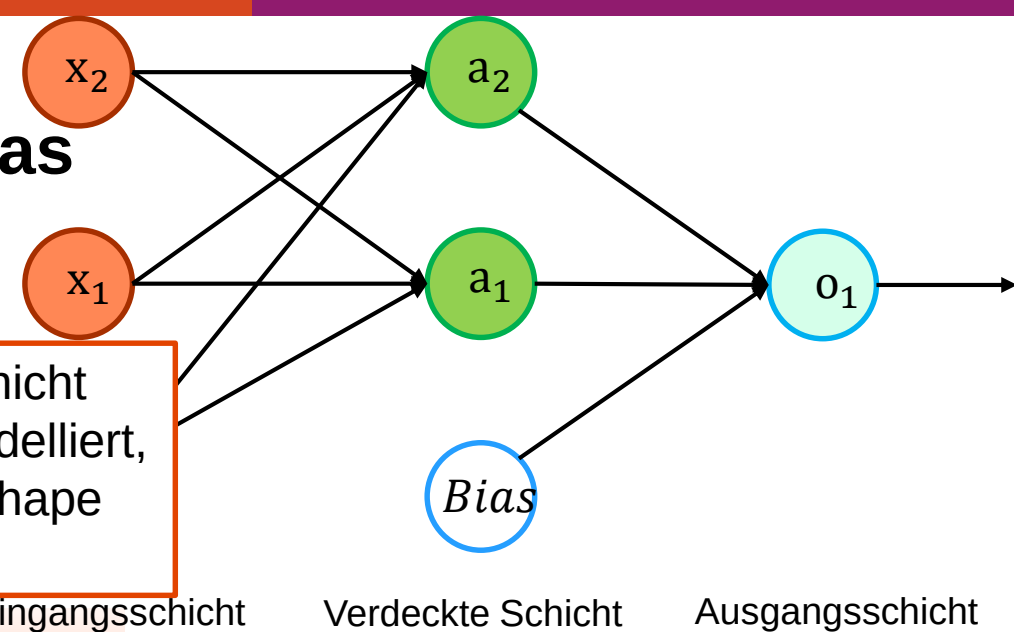
```
model.fit(x_train, y_train, epochs=500, verbose=0)
```

```
classes = (model.predict(x_test) > 0.5).astype("int32")
print("Predictions:", classes)
```

Eingabeschicht wird nicht als eigener Layer modelliert, sondern über input_shape definiert

Modell erstellen

SGD: Stochastic Gradient Descent



Neuronale Netze programmieren in keras

Das XOR-Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

```
model = Sequential()
model.add(Dense(units=2, activation='sigmoid', input_shape=(2,)))
model.add(Dense(units=1, activation='sigmoid'))
```

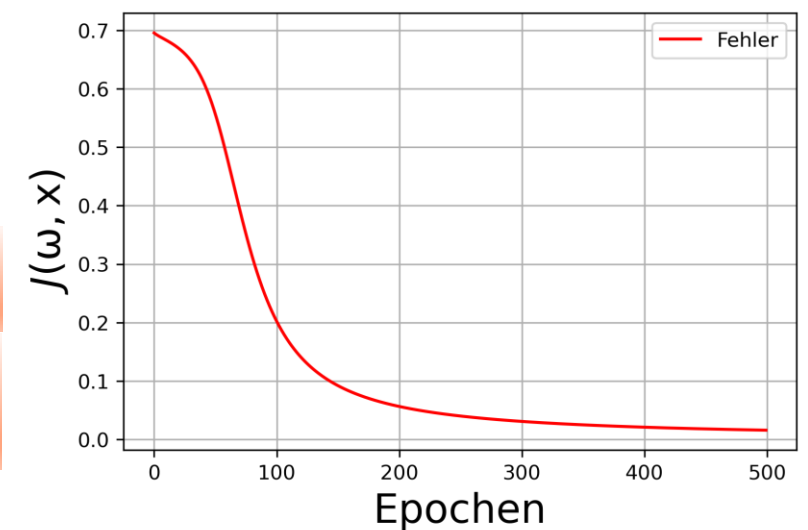
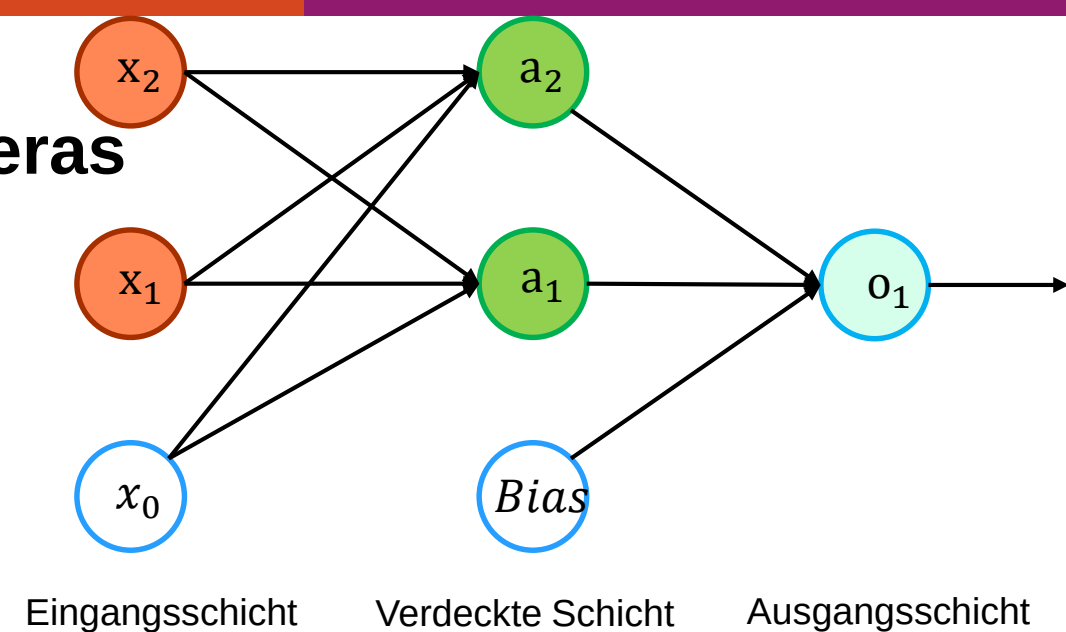
```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=500, verbose=0)
```

Modell trainieren

```
classes = (model.predict(x_test) > 0.5).astype("int32")
print("Predictions:", classes)
```

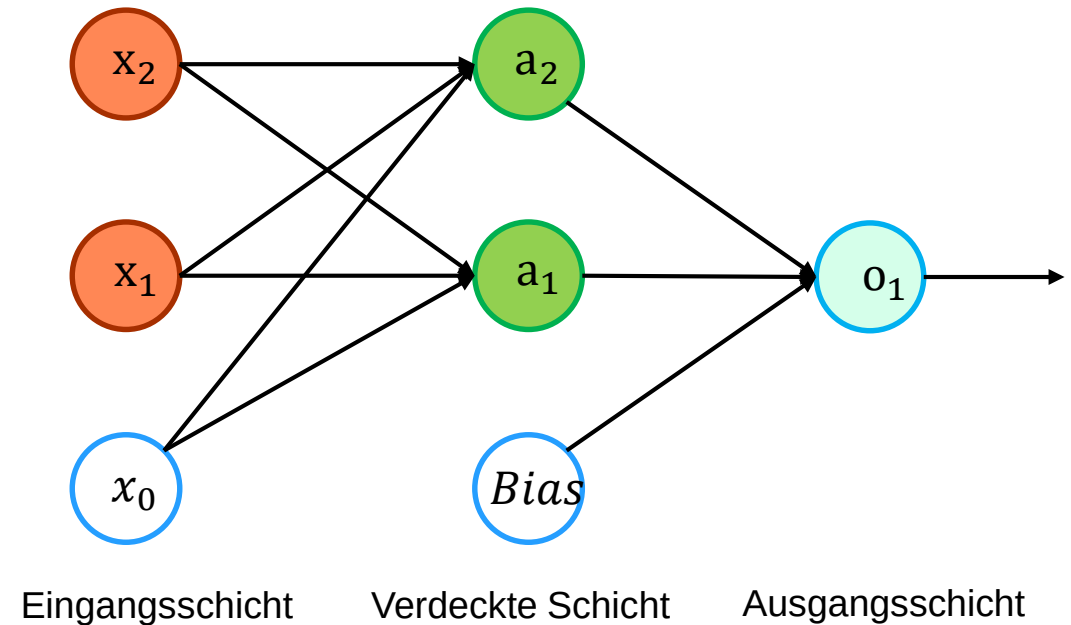
Modell testen



Neuronale Netze programmieren in keras

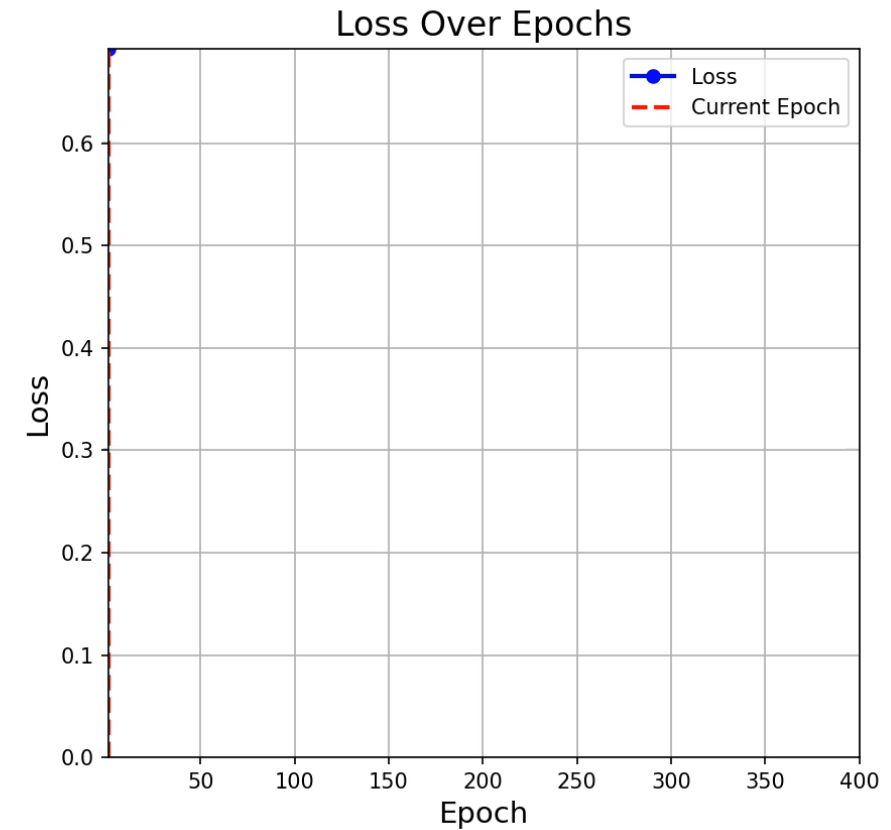
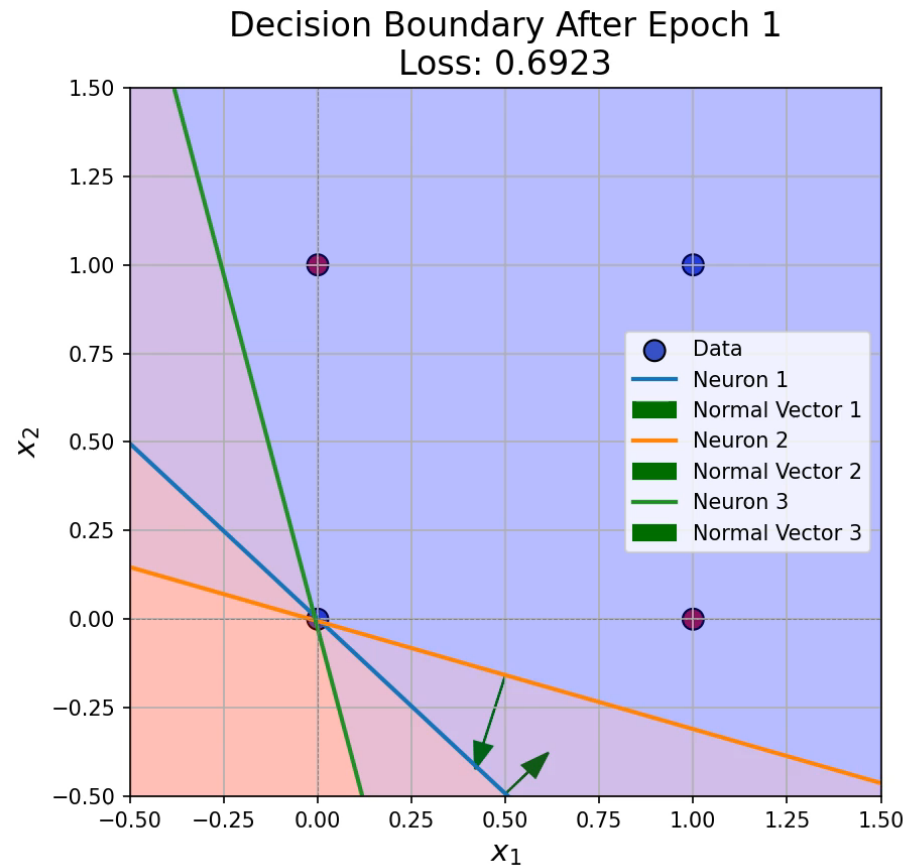
Das XOR-Beispiel - Übung

1. Gegeben Code mit einem Neuron ([01_xor_neuralnet1neuron.ipynb](#)), erweitern Sie den Code, sodass Sie drei Neurone haben.
 - 2 in verdeckter Schicht und 1 in Ausgangsschicht
2. Nutzen Sie auch mal drei Neurone in der verdeckten Schicht.
3. Fügen Sie eine zweite verdeckte Schicht hinzu.
4. Vergleichen Sie den Verlauf des Trainingsloss über die Anzahl Epochen für die drei Lösungen



Künstliche Neuronale Netze - KNN

Das XOR-Problem – 3 Neurone im Hidden Layer



Künstliche Neuronale Netze - KNN

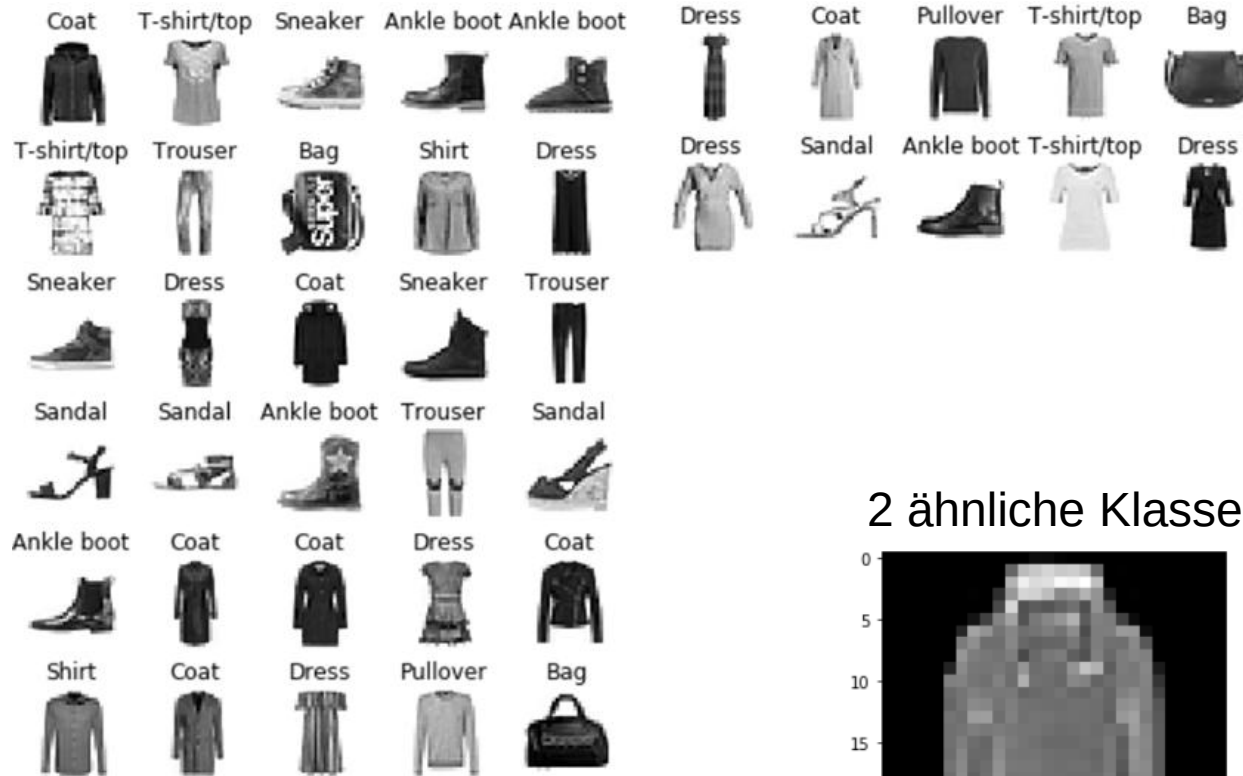
Einfaches Beispiel

Binäres Klassifikationsproblem

- Welchen Einfluss hat die Anzahl Schichten auf die Güte des neuronalen Netzes?
- Welchen Einfluss hat die Aktivierungsfunktion auf die Güte des neuronalen Netzes?
- Welchen Einfluss hat der Optimierungsalgorithmus, um möglichst schnell ein gutes neuronales Netz zu erhalten?

Künstliche Neuronale Netze - KNN

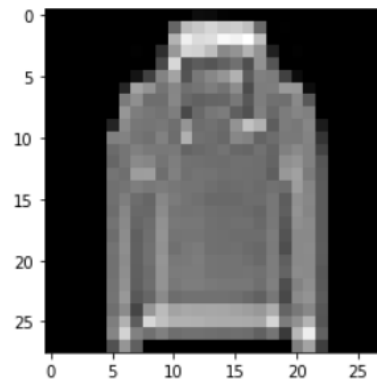
Beispiel Bildklassifizierer: Fashion MNIST



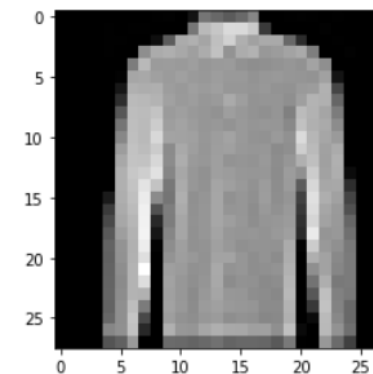
Fashion-MNIST: Zalando-Artikel

- 28x28 Graustufenbilder
- 60.000 Bilder
- 10 Klassen:
 - T-Shirt/Top, Hose, Pullover, Kleid, Mantel, Sandale, Hemd, Sneaker, Tasche, Stiefelette

2 ähnliche Klassen für die binäre Klassifizierung



Pullover
5507 Bilder



Hemd
5496 Bilder

Neuronale Netze programmieren in keras

Code für das Fashion MNIST Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test)= load_data()
```

```
model = Sequential()
```

Flatten wandelt Matrix in Vektor um

```
model.add(Flatten(input_shape=[28,28]))
```

```
model.add(Dense(units=1, activation='sigmoid'))
```

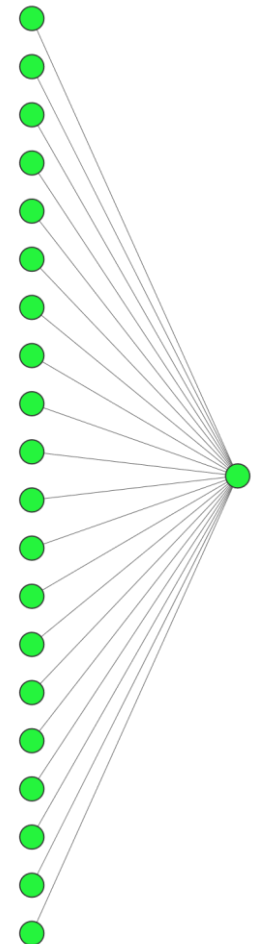
Modell erstellen

```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=100)
```

Modell trainieren

Eingabeschicht:
 $28 \times 28 = 784$
Neuronen



Neuronale Netze programmieren in keras

Code für das Fashion MNIST Beis

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

```
model = Sequential()
```

```
model.add(Flatten(input_shape=[28,28]))
```

```
model.add(Dense(units=1, activation='sigmoid'))
```

```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=100)
```

```
model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 1)	785

```

Total params: 785
Trainable params: 785
Non-trainable params: 0

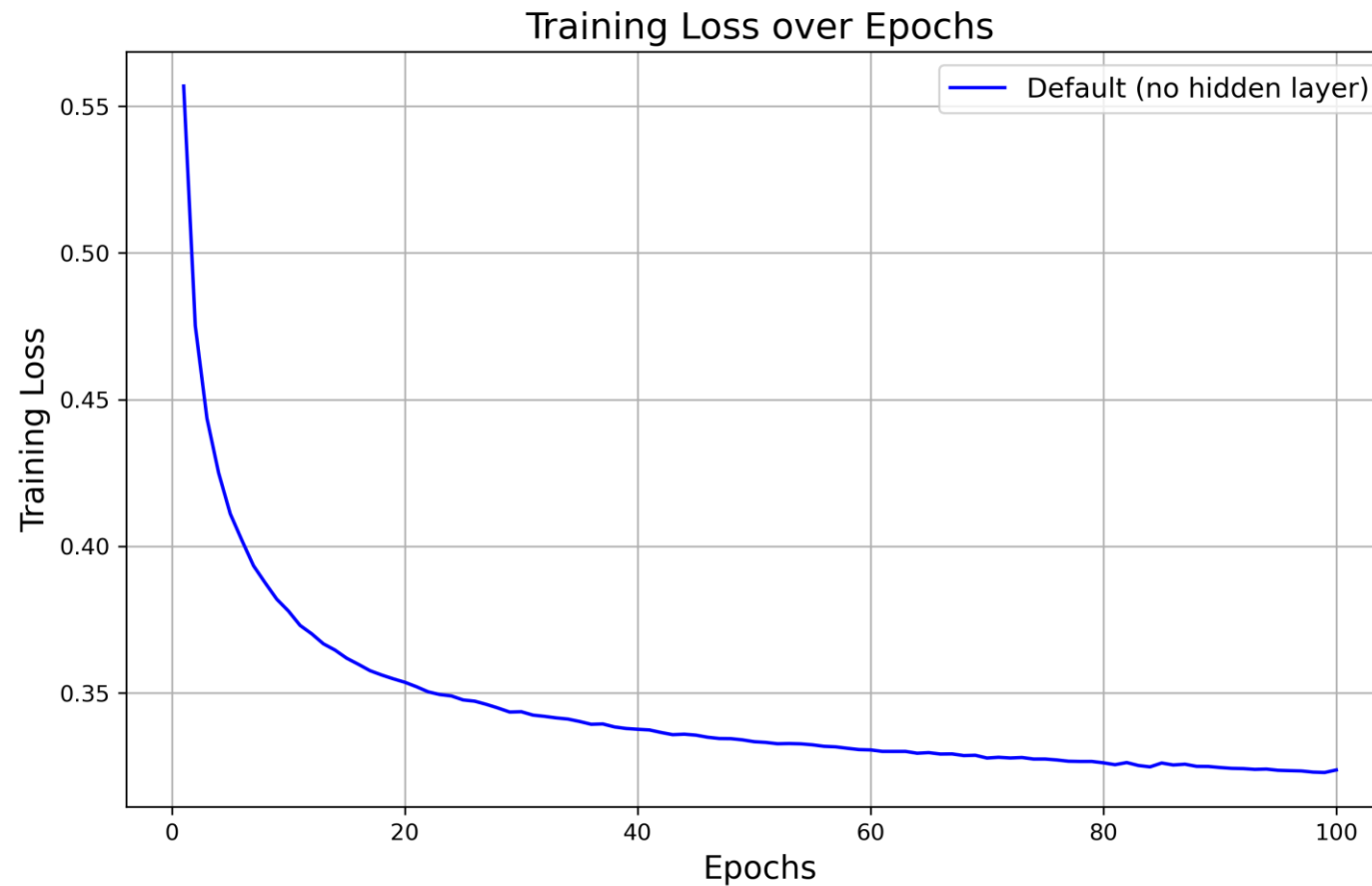
```

Modell erstellen

Modell trainieren

Künstliche Neuronale Netze - KNN

Beispiel Bildklassifizierer: Trainingsergebnisse von neuronalem Netz mit 1 Layer



Künstliche Neuronale Netze - KNN

Vorwärtspropagation eines Mehrschichtigen neuronalen Netzes

Vorwärtspropagation (mit N verdeckten Schichten):

$$a_j^{[1]} = \sigma^{[1]} \left(\sum_i \omega_{ji}^{[1]} \cdot x_i \right)$$

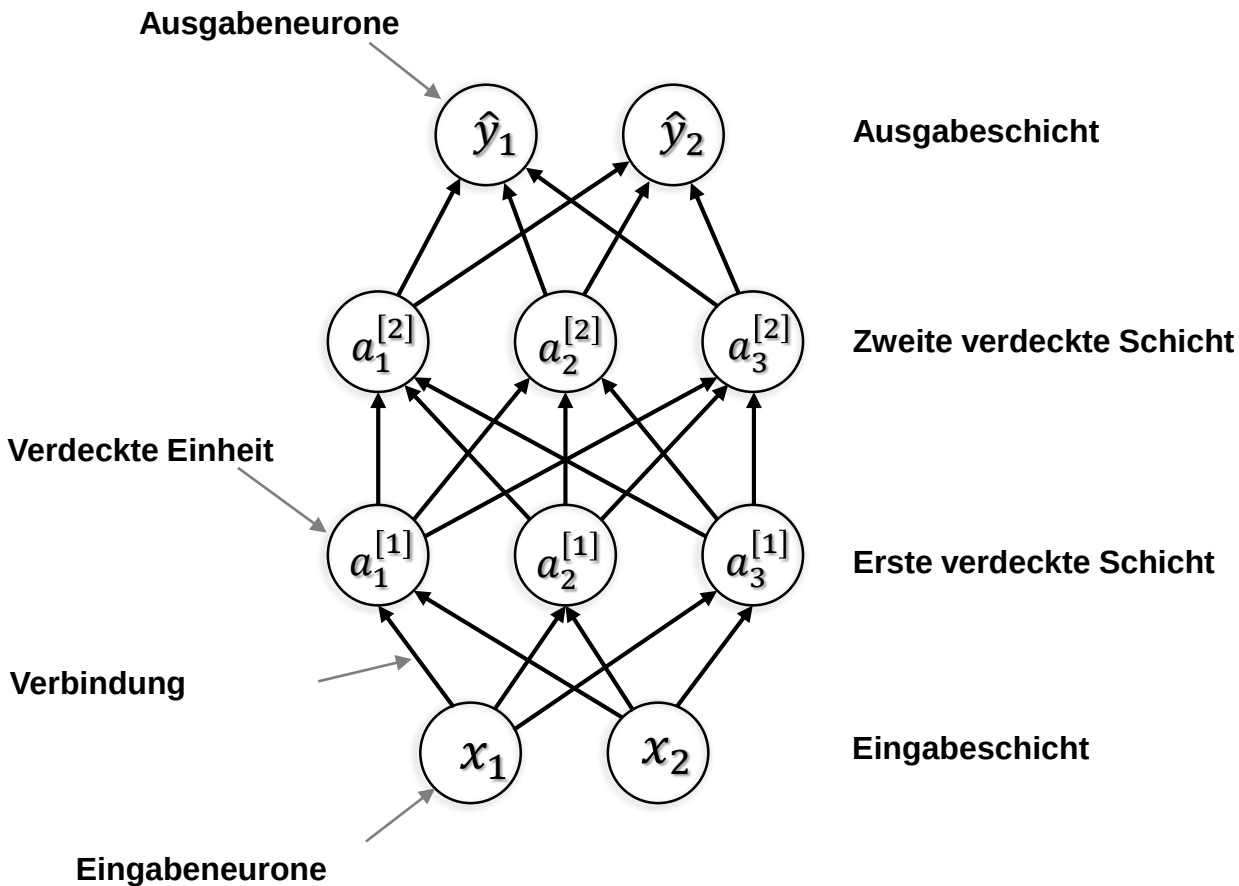
$$a_j^{[\ell]} = \sigma^{[\ell]} \left(\sum_i \omega_{ji}^{[\ell]} \cdot a_i^{[\ell-1]} \right), \ell = 2, \dots, N$$

$$\hat{y}_j = \sigma^{[N+1]} \left(\sum_i \omega_{ji}^{[N+1]} \cdot a_i^{[N]} \right)$$

$a_j^{[\ell]}$: Aktivierungsfunktion des j -ten Neurons der ℓ -ten verdeckten Schicht

$\omega_{ji}^{[\ell]}$: Gewicht des i -ten Eingangs des j -ten Neurons der ℓ -ten verdeckten Schicht

$\hat{y}_j$: Vorhersage des j -ten Ausgabeneurons



Künstliche Neuronale Netze - KNN

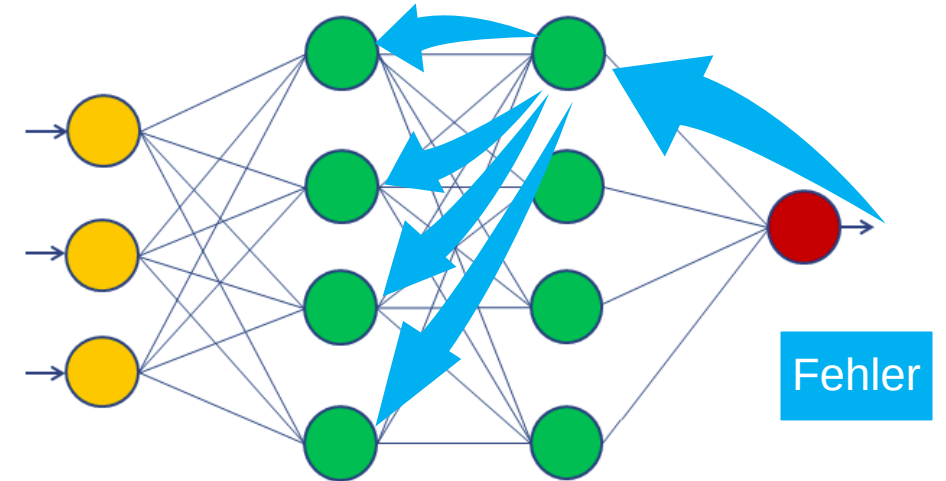
Trainieren neuronaler Netze

Trainieren: Minimierung der Kostenfunktion $J(\omega)$, die Fehler zwischen Vorhersage und Label berechnet

Backpropagation

- Rückführung des Fehlers durch das gesamte Netz
- Anteilige Aufteilung des Fehlers auf alle Neuronen

$$\nabla_{\omega} J(\omega) = \begin{pmatrix} \frac{\partial J(\omega)}{\partial \omega_1} \\ \frac{\partial J(\omega)}{\partial \omega_2} \\ \frac{\partial J(\omega)}{\partial \omega_3} \end{pmatrix}$$



Künstliche Neuronale Netze - KNN

Trainieren neuronaler Netze

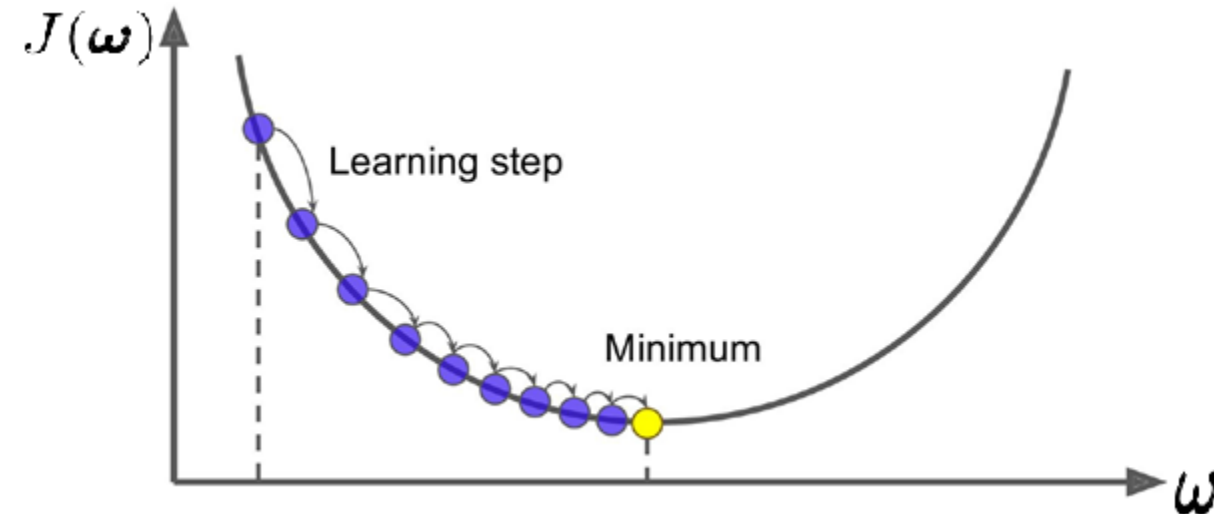
Minimierung der Kostenfunktion durch

Gradientenabstieg (Optimierungsalgorithmus)

- Aktualisierung der Parameter (Gewichte, Bias)
- Entgegen der Richtung des Gradienten der Kostenfunktion
- Schrittweite η wird als Lernrate bezeichnet

$$\omega \leftarrow \omega - \eta \nabla_{\omega} J(\omega) \quad \nabla_{\omega} J(\omega) = \begin{pmatrix} \frac{\partial J(\omega)}{\partial \omega_1} \\ \frac{\partial J(\omega)}{\partial \omega_2} \\ \frac{\partial J(\omega)}{\partial \omega_3} \end{pmatrix}$$

Gradient zeigt in Richtung des steilsten Anstiegs



Künstliche Neuronale Netze - KNN

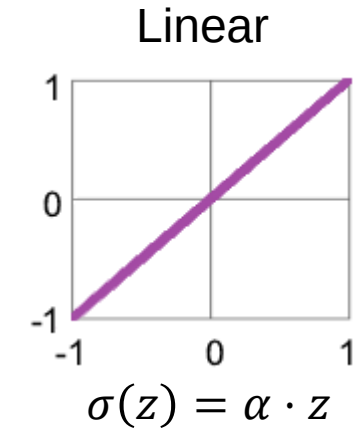
Beispiel Bildklassifizierer: Vergleich von zwei Aktivierungsfunktionen

Mehrlagiges Netz mit einer Eingangsschicht (Flatten), einer verdeckten Schicht (Dense und 300 Neuronen) und einer Ausgangsschicht (Dense, Sigmoid):

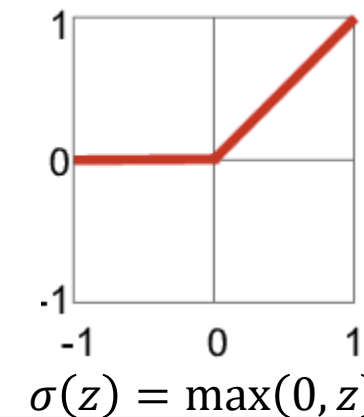
- verdeckte Schicht mit linearer Aktivierung
Einführung des **Hyperparameters** α
- verdeckte Schicht mit ReLU (Rectifier Linear Unit) Aktivierung

```
model_linear.add(Flatten(input_shape=[28, 28]))
model_linear.add(Dense(units=300, activation='linear'))
model_linear.add(Dense(units=1, activation='sigmoid'))
```

```
model_relu.add(Flatten(input_shape=[28, 28]))
model_relu.add(Dense(units=300, activation='relu'))
model_relu.add(Dense(units=1, activation='sigmoid'))
```



Rectified Linear Unit (ReLU)

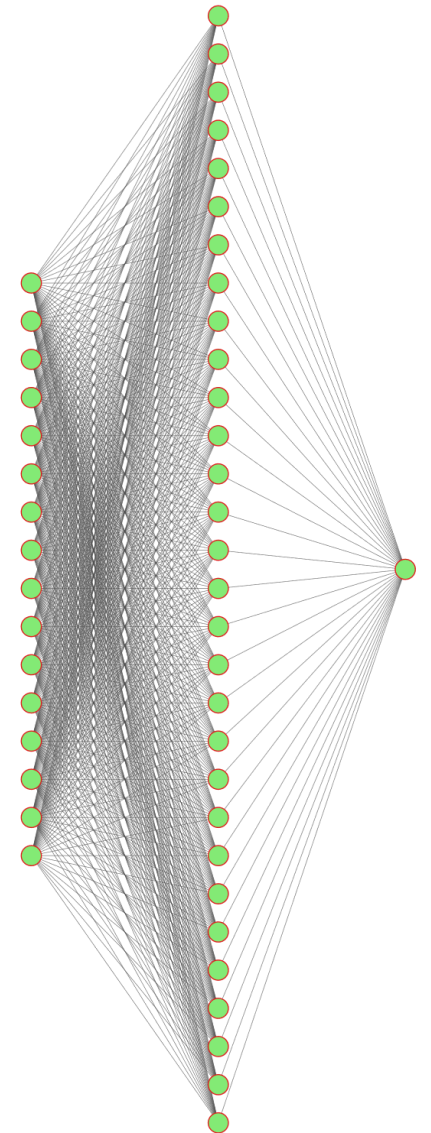


Künstliche Neuronale Netze - KNN

Beispiel Bildklassifizierer: Vergleich von zwei Aktivierungsfunktionen

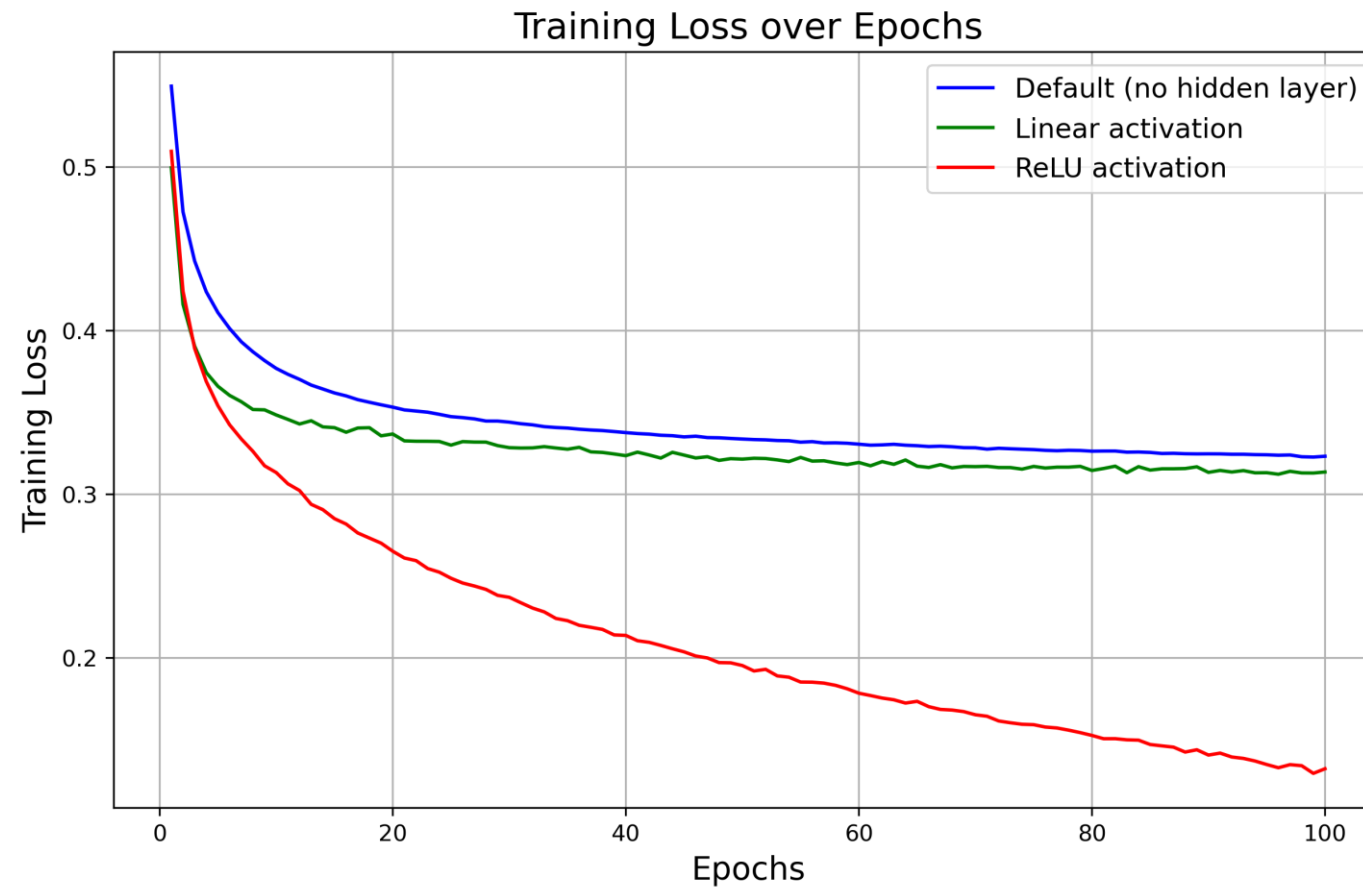
```
model_relu.summary()
```

Layer (type)	Output Shape	Param #
=====		
flatten_2 (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 300)	235500
dense_4 (Dense)	(None, 1)	301
=====		
Total params: 235,801		
Trainable params: 235,801		
Non-trainable params: 0		



Künstliche Neuronale Netze - KNN

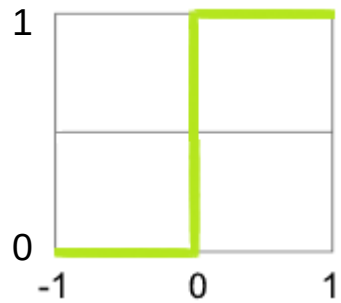
Beispiel Bildklassifizierer: Vergleich von zwei Aktivierungsfunktionen



Künstliche Neuronale Netze - KNN

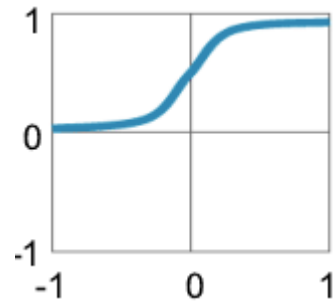
Aktivierungsfunktionen

Binäre Stufenfunktion



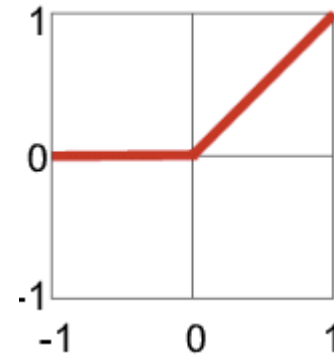
$$f(z) = \begin{cases} 0 & \text{falls } z < 0 \\ 1 & \text{sonst} \end{cases}$$

Sigmoid



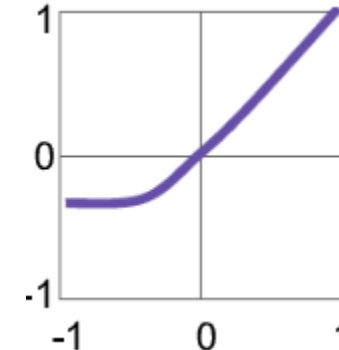
$$f(z) = \frac{1}{1 + e^{-z}}$$

Rectified Linear Unit (ReLU)



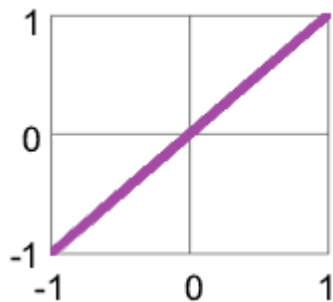
$$f(z) = \max(0, z)$$

Scaled Exp. LU (SELU)



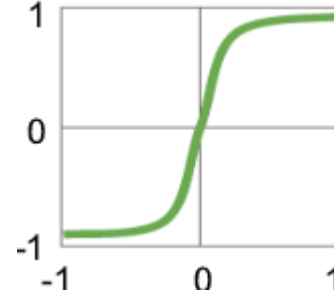
$$f(z) = \lambda \begin{cases} \alpha(e^z - 1) & \text{falls } z < 0 \\ z & \text{sonst} \end{cases}$$

Linear



$$f(z) = \alpha z$$

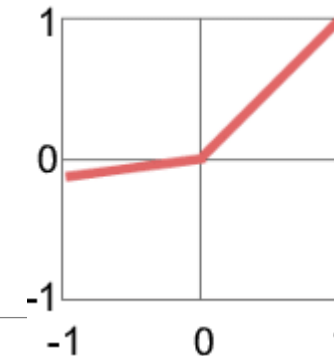
Tangens hyperbolicus



$$f(z) = \tanh(z)$$

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Leaky ReLU



$$f(z) = \max(\alpha z, z)$$

Softmax

$$f(z_k) = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_c e^{z_c}}$$

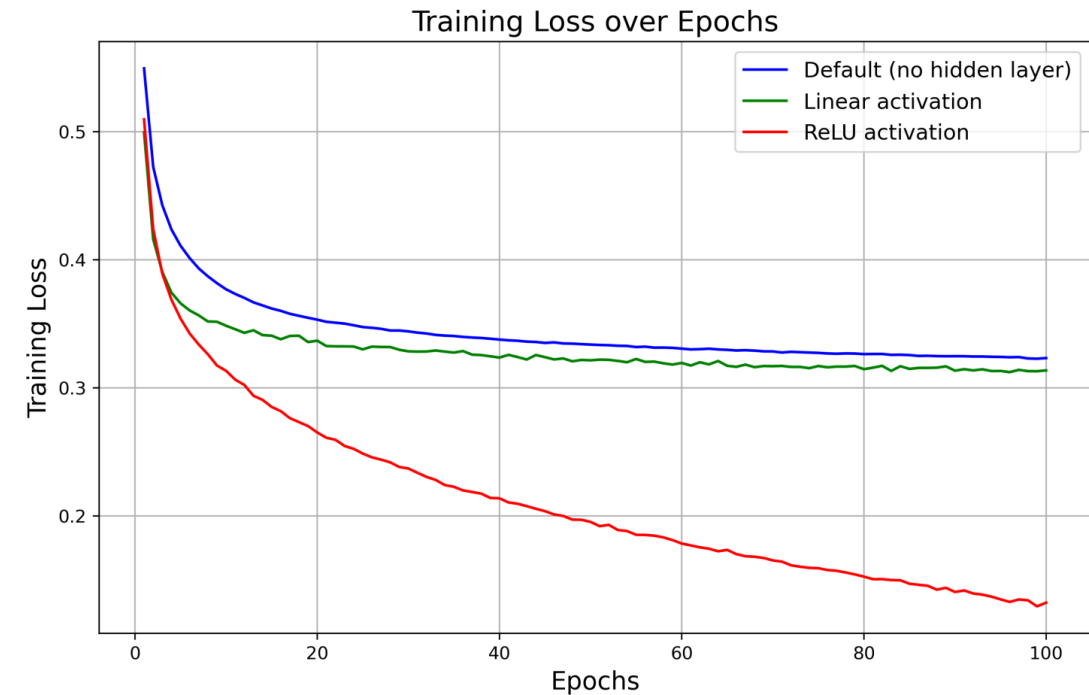
Künstliche Neuronale Netze - KNN

Übung Bildklassifizierer: Vergleich von Aktivierungsfunktionen

- Probieren Sie verschiedene Aktivierungsfunktionen aus und schauen Sie sich den Trainingsloss über die Epochen an.

- Nutzen Sie:

[02_fashion_mnist_binary.ipynb](#)



Deep Learning - Zutaten

Modellarchitektur

- Anzahl Schichten
- Aktivierungsfunktion

Kostenfunktion

Definiert was ein gutes neuronales Netz ist

Optimierungsalgorithmus

Minimiert die Kostenfunktion, indem es Gewichte des NNs variiert

Training des NNs

Minimierung der Kostenfunktion

Künstliche Neuronale Netze - KNN

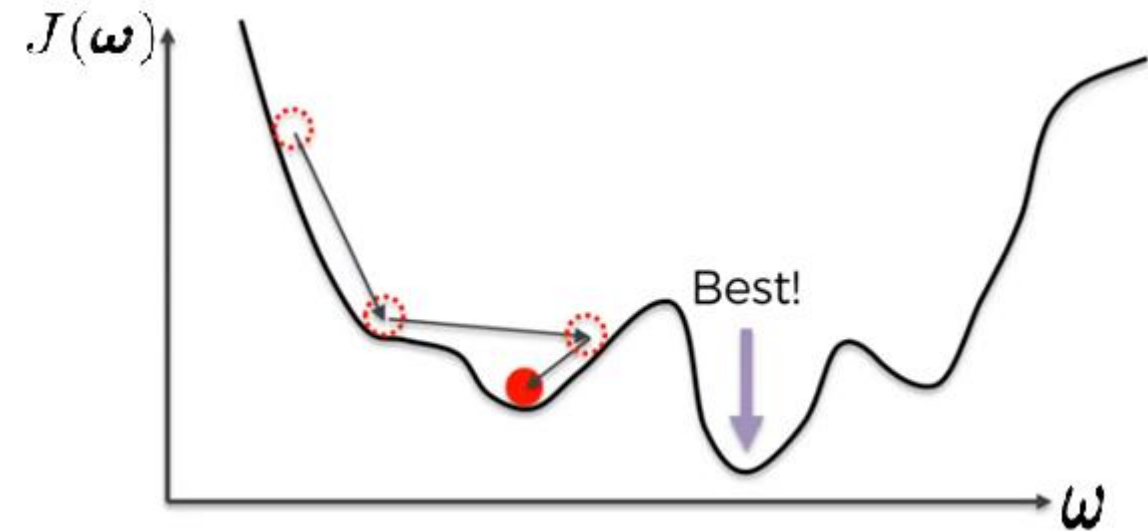
Gradient Descent minimiert Kostenfunktion

Kostenfunktion für binäres Klassifikationsproblem:

- Binäre Kreuzentropie (binary cross entropy)

$$\omega \leftarrow \omega - \eta \cdot \frac{\partial J(\omega, x)}{\partial \omega} = \omega - \eta \cdot \nabla_{\omega} J(\omega, x)$$

```
model.compile(loss='binary_crossentropy', optimizer=SGD())
```



$$J(\omega; \mathbf{X}) = -\frac{1}{N} \sum_{i=0}^N \left(y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right) \quad \text{mit} \quad \left(\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \right)$$

Künstliche Neuronale Netze - KNN

Gradient Descent

Vanilla Gradient Descent (Batch Gradient Descent)

- Iteriert über alle Trainingsinstanzen
- Eignet sich nur bei konvexen Funktionen
- Ungeeignet für große Trainingsdatensätze

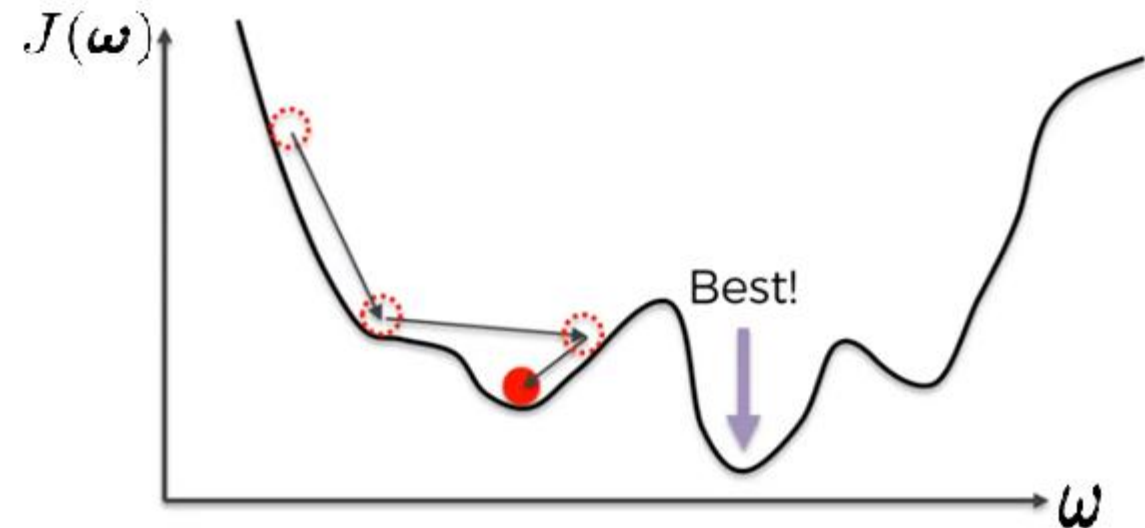
Stochastic Gradient Descent (SGD)

- Nimmt eine zufällige Trainingsinstanz
- Flucht aus lokalen Minima möglich

Mini-Batch Gradient Descent

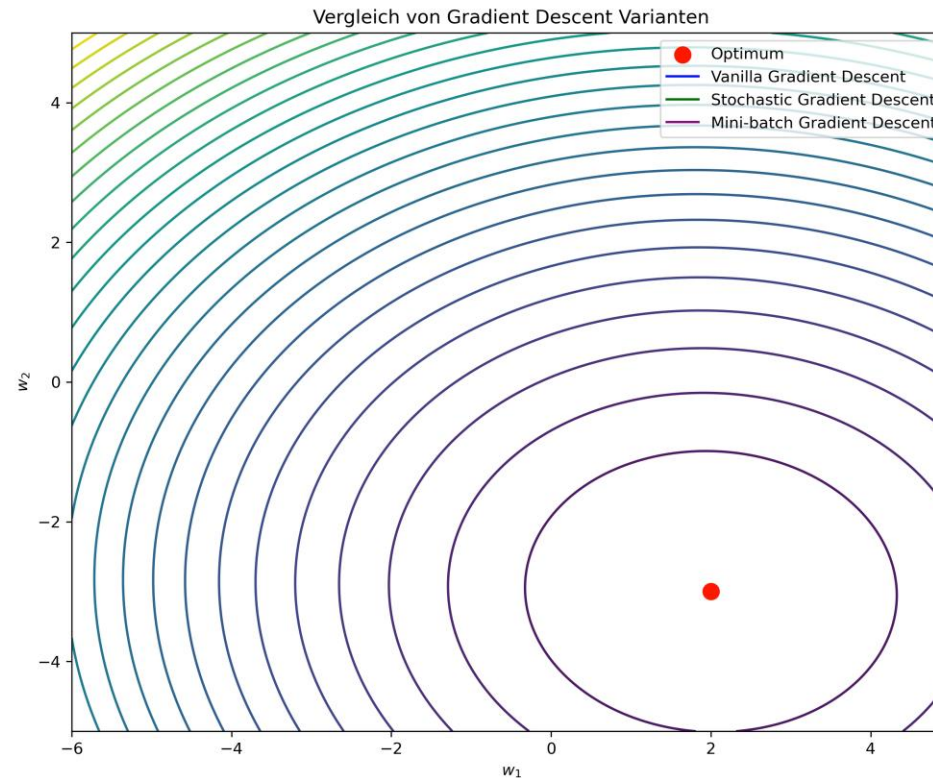
- Nutzt zufällige Mini-Batches
- Mini-Batch: Teil des Trainingsdatensatzes
- Nutzt die Vorteile beider vorherigen Methoden
- Große Datensätze: Schnelleres Training

- $\omega \leftarrow \omega - \eta \cdot \frac{\partial J(\omega, x)}{\partial \omega} = \omega - \eta \cdot \nabla_{\omega} J(\omega, x)$
- $\omega \leftarrow \omega - \eta \cdot \nabla_{\omega} J(\omega, x^{(i)}; y^{(i)})$
- $\omega \leftarrow \omega - \eta \cdot \nabla_{\omega} J(\omega, x^{(i:i+n)}; y^{(i:i+n)})$



Künstliche Neuronale Netze - KNN

Gradient Descent



Künstliche Neuronale Netze - KNN

Beispiel Bildklassifizierer: Vergleich von drei Optimierungsalgorithmen

Mehrlagiges Netz mit einer Eingangsschicht (Flatten), einer verdeckten Schicht (Dense) und einer Ausgangsschicht (Dense):

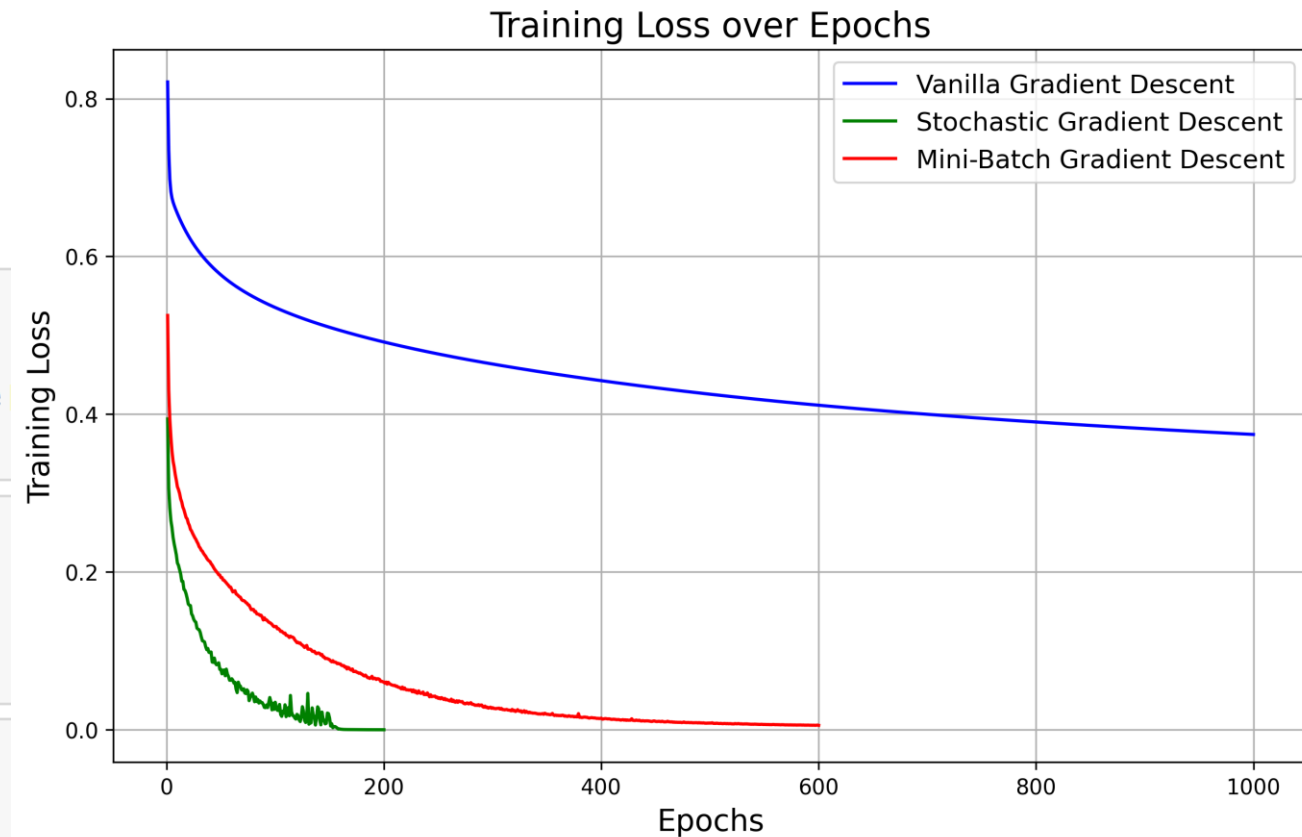
- Vanilla Gradient Descent (1000, 35.14 s)
- Stochastic Gradient Descent (200, 1713.81 s)
- Mini-batch Gradient Descent (600, 216.22 s)

```
history_VGD = model_VGD.fit(X_train_two,
                             y_train_two,
                             batch_size=X_train_two.shape
                             epochs=1000)
```

```
history_SGD = model_SGD.fit(X_train_two,
                             y_train_two,
                             batch_size=1,
                             epochs=200)
```

```
history_BGD = model_BGD.fit(X_train_two,
                             y_train_two,
                             epochs=600)
```

Standardwert: 32



Künstliche Neuronale Netze - KNN

Einfaches Beispiel - Ergebnisse

Binäres Klassifikationsproblem

- Welchen Einfluss hat die Anzahl Schichten auf die Güte des neuronalen Netzes?
 - Höhere Anzahl Schichten = Mehr Gewichte = Mehr Freiheitsgrade
 - → kleinerer finaler Loss auf den Trainingsdaten
- Welchen Einfluss hat die Aktivierungsfunktion auf die Güte des neuronalen Netzes?
 - Die Aktivierungsfunktion ist das einzig Nichtlineare im neuronalen Netz
 - Aktivierungsfunktion hat auch einen erheblichen Einfluss auf die Konvergenz des Gradientenabstiegs (dazu später mehr)
- Welchen Einfluss hat der Optimierungsalgorithmus, um möglichst schnell ein gutes neuronales Netz zu erhalten?
 - Mini-Batch Gradient Descent beschleunigt das Training sehr
 - Später werden wir weitere Optimierungsalgorithmen kennenlernen

Künstliche Neuronale Netze - KNN

Beispiel Bildklassifizierer: Fashion MNIST



Fashion-MNIST: Zalando-Artikel

- 28x28 Graustufenbilder
- 60.000 Bilder
- 10 Klassen:
 - T-Shirt/Top, Hose, Pullover, Kleid, Mantel, Sandale, Hemd, Sneaker, Tasche, Stiefelette

Benötigte Änderungen:

- Aktivierungsfunktion der letzten Schicht
- Kostenfunktion

Künstliche Neuronale Netze - KNN

Trainieren neuronaler Netze – Kostenfunktion $J(\omega)$

Kostenfunktion $J(\omega; \mathbf{X})$: misst die Abweichung zwischen den Ausgängen $\hat{\mathbf{y}}^{(i)} = h(\mathbf{x}^{(i)})$ und den entsprechenden Sollwerten $\mathbf{y}^{(i)}$

- Mittlere Abweichung (*mean squared error*):

$$J(\omega; \mathbf{X}) = \frac{1}{N} \sum_{i=0}^N (\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)})^2$$

Künstliche Neuronale Netze - KNN

Trainieren neuronaler Netze – Kostenfunktion $J(\omega)$

Kostenfunktion $J(\omega; \mathbf{X})$: misst die Abweichung zwischen den Ausgängen $\hat{\mathbf{y}}^{(i)} = h(\mathbf{x}^{(i)})$ und den entsprechenden Sollwerten $\mathbf{y}^{(i)}$

- Kreuzentropie / kategoriale Kreuzentropie (*cross entropy / categorical cross entropy*):
 - Softmax transformiert K-dimensionalen Vektor $\mathbf{z}$ in den Wertebereich (0,1) mit Summe 1

$$J(\omega; \mathbf{X}) = -\frac{1}{N} \sum_{i=0}^N \sum_{k=0}^K y_k^{(i)} \log(\hat{p}_k^{(i)}) \quad \text{mit} \quad \left(\hat{p}_k = \sigma(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=0}^K e^{z_j}} \right)$$

The diagram shows a vector $\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$ being processed by a box labeled $\frac{e^{z_k}}{\sum_{j=0}^K e^{z_j}}$ to produce a probability vector $\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$.

- Binäre Kreuzentropie (2 Klassen: $K=1$) (*binary cross entropy*):

$$J(\omega; \mathbf{X}) = -\frac{1}{N} \sum_{i=0}^N \left(y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right) \quad \text{mit} \quad \left(\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \right)$$

Neuronale Netze programmieren in keras

Code für das Fashion MNIST Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

Daten importieren

```
model = Sequential()
```

```
model.add(Flatten(input_shape=[28,28]))
model.add(Dense(units=300, activation='relu'))
model.add(Dense(units=10, activation='softmax'))
```

Modell erstellen

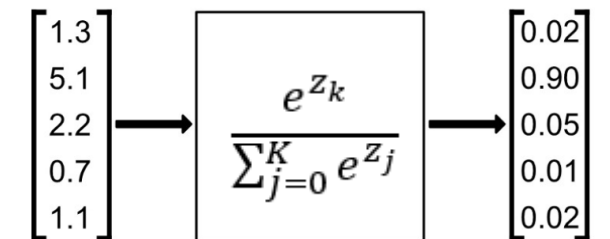
```
model.compile(loss='sparse_categorical_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=100)
```

Modell trainieren

Softmax Aktivierungsfunktion:

Garantiert, dass die Ausgabe des neuronalen Netzes ein normalisierter K-dimensionaler Vektor ist (hier K=10), d.h. ein Wahrscheinlichkeitsvektor ist.



Neuronale Netze programmieren in keras

Code für das Fashion MNIST Beispiel

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD
```

```
(x_train, y_train), (x_test, y_test) = load_data()
```

Daten importieren

```
model = Sequential()
```

```
model.add(Flatten(input_shape=[28,28]))
model.add(Dense(units=300, activation='relu'))
model.add(Dense(units=10, activation='softmax'))
```

Modell erstellen

```
model.compile(loss='sparse_categorical_crossentropy', optimizer=SGD())
```

```
model.fit(x_train, y_train, epochs=100)
```

Modell trainieren

sparse_categorical_crossentropy und *categorical_crossentropy*

Beispiel: 5 Samples mit 3 Klassen

- Klassen schließen sich gegenseitig aus

sparse_categorical_crossentropy:

$$y = \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \\ 0 \end{pmatrix}$$

categorical_crossentropy:

$$y = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Neuronale Netze programmieren in keras

Übung: Fashion MNIST Beispiel

1. Trainieren Sie das neuronale Netz unter Nutzung der „*sparse_categorical_crossentropy*“.
Nutzen Sie: [03_fashion_mnist_10_loss_fun.ipynb](#)
2. Ändern Sie die „loss function“ zu „*categorical_crossentropy*“ und passen Sie die Labels y so an, dass Sie die neue loss function nutzen können.
 - Siehe „load_data_full“ in `utils_fminst.py`
3. Trainieren Sie das neuronale Netz mit der neuen loss function. Vergleichen Sie die Ergebnisse mit denen aus dem ersten Experiment.

sparse_categorical_crossentropy und *categorical_crossentropy*

Beispiel: 5 Samples mit 3 Klassen

- Klassen schließen sich gegenseitig aus

sparse_categorical_crossentropy:

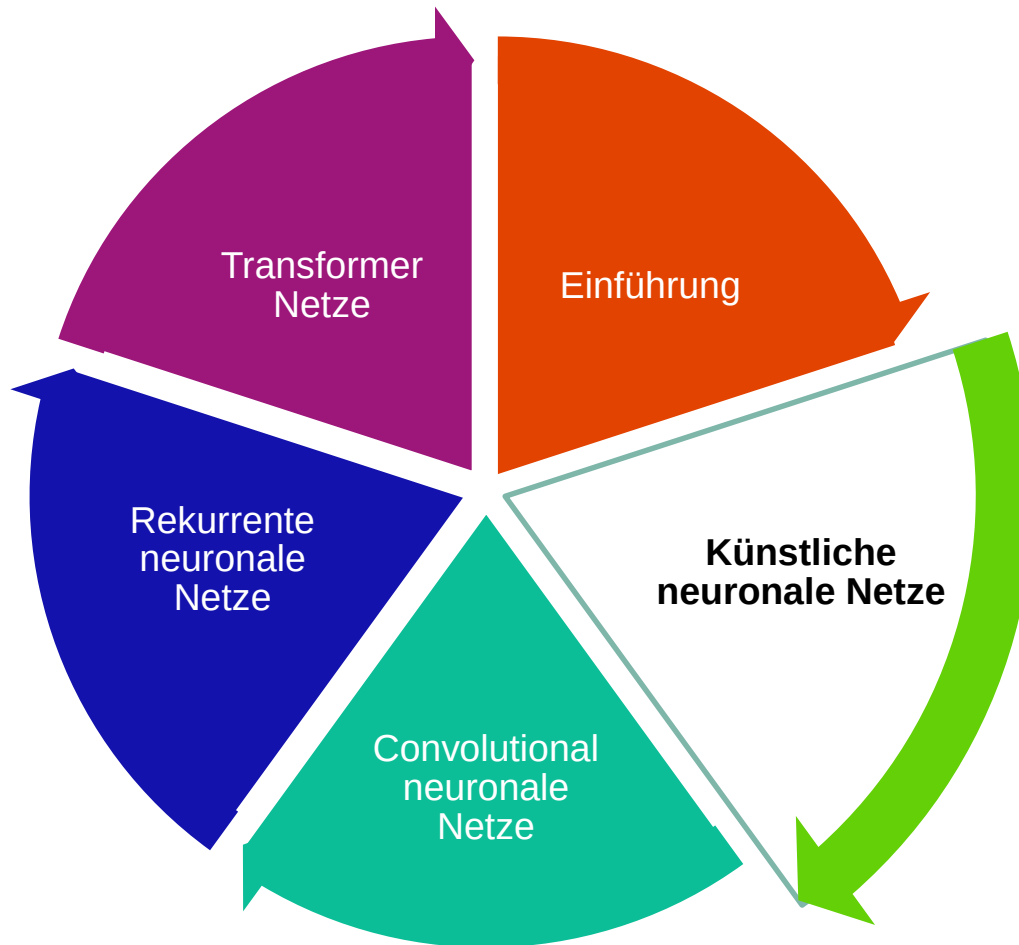
$$y = \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \\ 0 \end{pmatrix}$$

categorical_crossentropy:

$$y = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Deep Learning

Ausblick

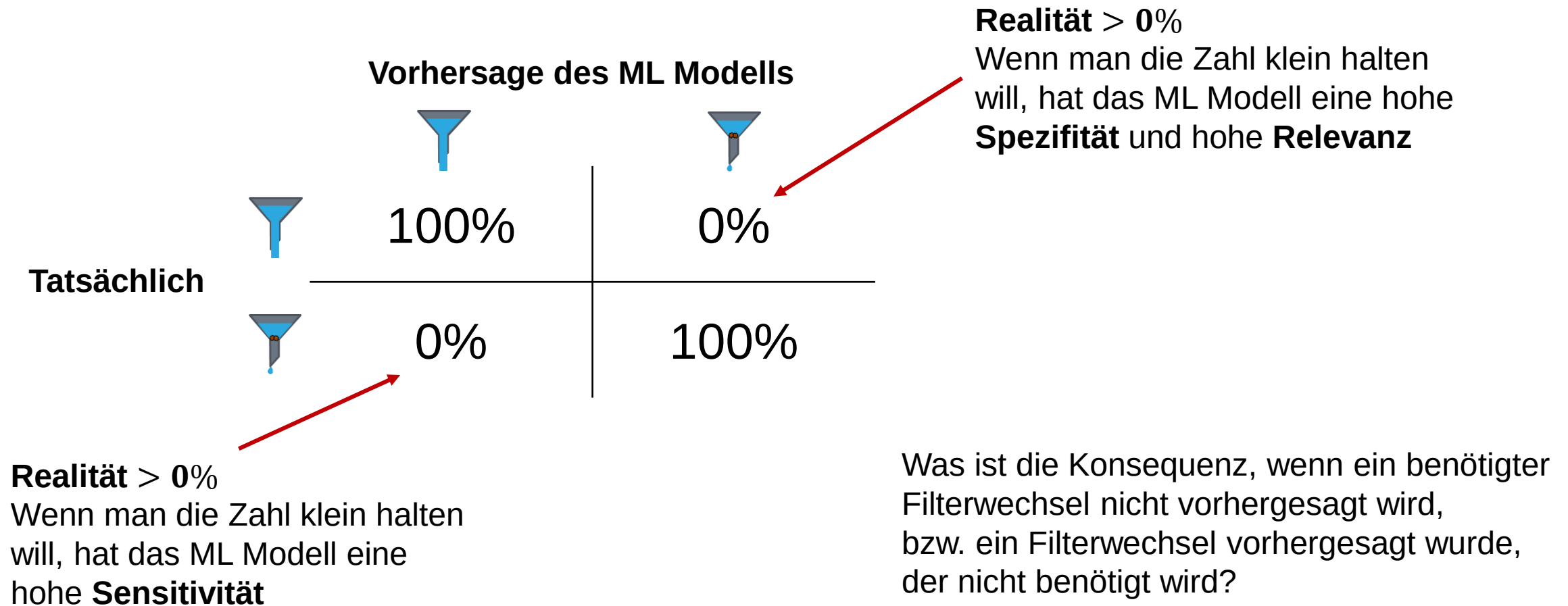


Künstliche neuronale Netze: Part 1

- Einführung
- Training neuronaler Netze
 - Optimierungsalgorithmus
 - Aktivierungsfunktionen
 - Kostenfunktion
- **Performancemetriken**

Künstliche Neuronale Netze - KNN

Performancemetriken – Wie gut ist mein neuronales Netz?



Künstliche Neuronale Netze - KNN

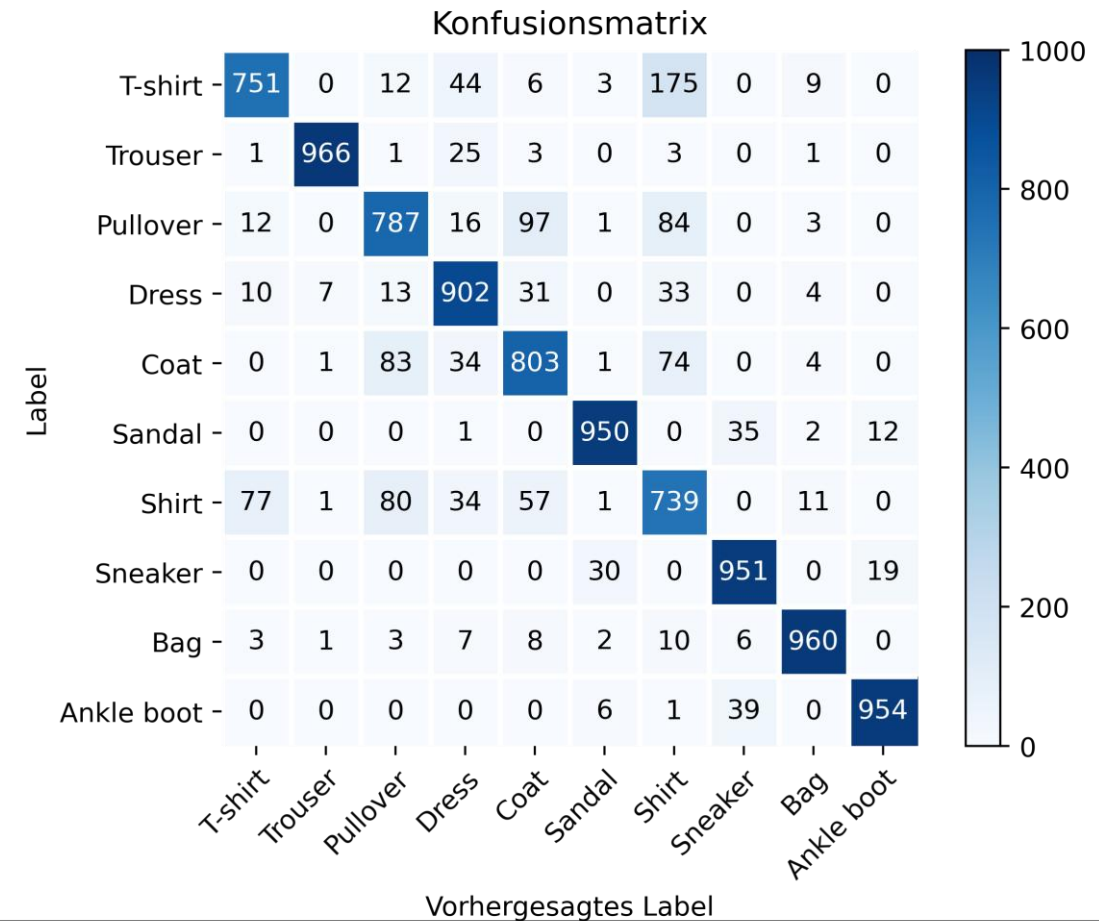
Beispiel Bildklassifizierer: Konfusionsmatrix

```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
# Berechnung der Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes)
```

```
array([[751,  0, 12, 44,  6,  3, 175,  0,  9,  0],
       [ 1, 966,  1, 25,  3,  0,  3,  0,  1,  0],
       [12,  0, 787, 16, 97,  1, 84,  0,  3,  0],
       [10,  7, 13, 902, 31,  0, 33,  0,  4,  0],
       [ 0,  1, 83, 34, 803,  1, 74,  0,  4,  0],
       [ 0,  0,  0,  1,  0, 950,  0, 35,  2, 12],
       [77,  1, 80, 34, 57,  1, 739,  0, 11,  0],
       [ 0,  0,  0,  0,  0, 30,  0, 951,  0, 19],
       [ 3,  1,  3,  7,  8,  2, 10,  6, 960,  0],
       [ 0,  0,  0,  0,  0,  6,  1, 39,  0, 954]])
```

S.:
[04_fashion_mnist_10_confusion_matrix.ipynb](#)



Künstliche Neuronale Netze - KNN

Fragen II

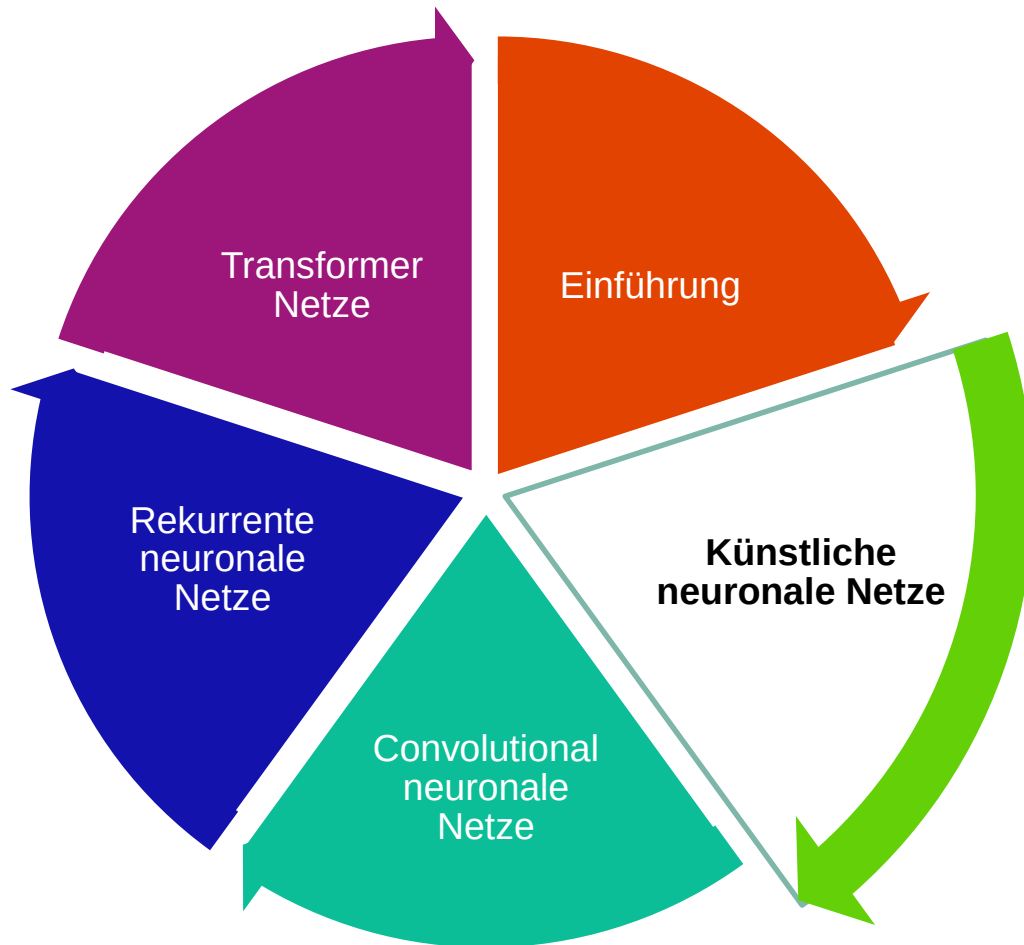
Was ist wahr in Bezug auf die Rückwärtspropagation?

- Der Fehler in der Ausgabe wird nur rückwärts propagiert, um Gewichtsaktualisierungen festzulegen
- Ein Lernalgorithmus für neuronale Netze mit nur einer Feedforward-Schicht
- Lokale Minima und langsame Konvergenz können die Rückwärtspropagation einschränken

Basiert das Lernen der Rückpropagation auf einem Gradientenabstieg entlang der Fehleroberfläche?

Deep Learning

Ausblick



Künstliche neuronale Netze: Part 2

- Trainings-, Test- und Validierungsdaten
- Regularisierung
 - **Early Stopping**
 - ℓ_1 & ℓ_2 -Regularisierung
 - **Dropout**
 - Gewichtsinitialisierung
 - Batch Normalisierung
- Optimierungsalgorithmen

Künstliche Neuronale Netze - KNN

Trainings-, Validierungs- und Testdaten

Split des Datensatzes in Trainings-, Validierungs- und Testdaten:

- z.B. mit einem Verhältnis von 75:10:15 (Es müssen disjunkte Mengen sein)
- Trainiere das neuronale Netz mit den Trainingsdaten
- Validiere das neuronale Netz mit den Validierungsdaten
- Endkontrolle auf den Testdaten (Test darf nur einmal erfolgen)

```
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]  
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
```

Was heißt validieren?

- Neuronale Netze haben Hyperparameter (Anzahl Schichten und Neuronen, Initialisierung der Gewichte, Datenvorverarbeitung, ...), welche eingestellt werden müssen
- Wann sollte ich das iterative Training beenden? (s. nächste Folie)

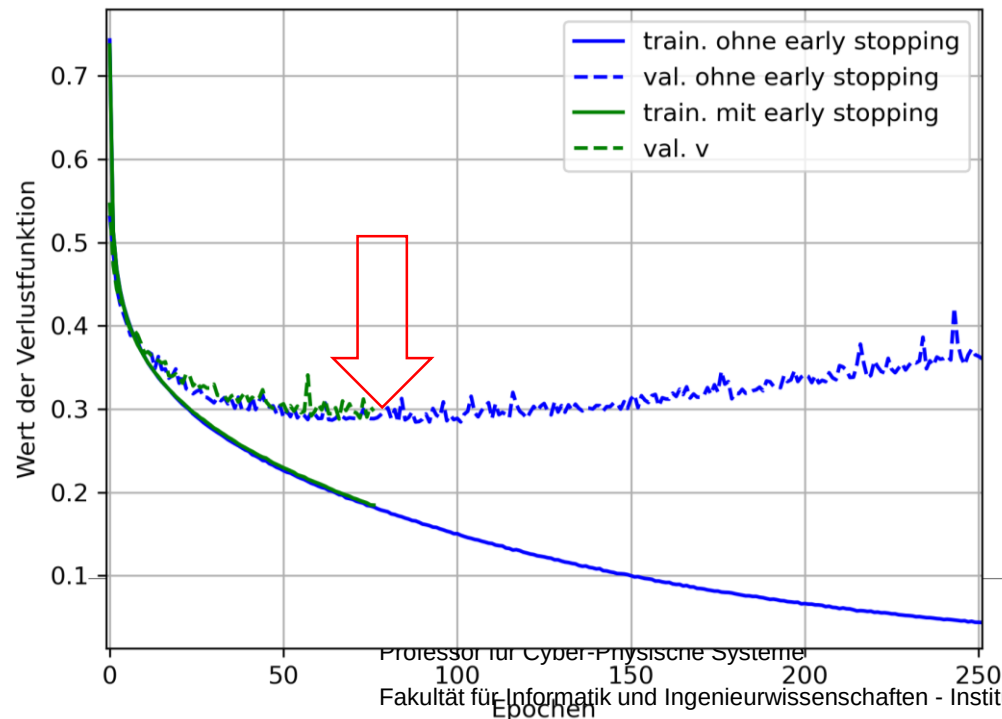
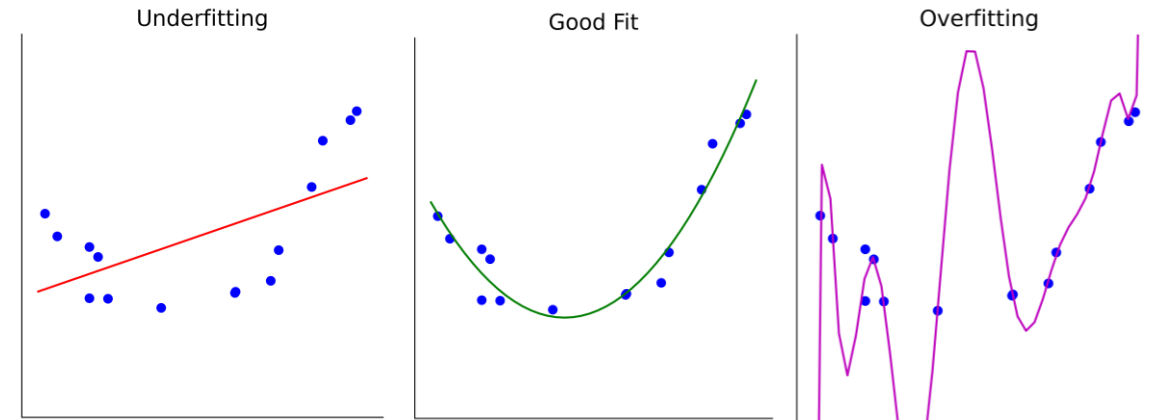
Künstliche Neuronale Netze - KNN

Regularisierung

Problemstellung:

Overfitting auf Trainingsdaten $\longrightarrow$ **Regularisierung**

- Early Stopping
Ermittelt das beste Modell anhand des Validierungsfehlers



```
history_early_stopping = model_early_stopping.fit(
    X_train,
    y_train,
    epochs=400,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        min_delta=1e-3,
        patience=10,
        restore_best_weights=True)],
    validation_data=(X_valid, y_valid))
```

Quelle: Patrick S., Hahn-Schickard

Künstliche Neuronale Netze - KNN

Regularisierung

ℓ_1 & ℓ_2 - Regularisierung

- Erweitern der Kostenfunktion um ein Regularisierungsterm $\Omega(\mathbf{W})$

$$\ell_2 : \Omega(\mathbf{W}) = \frac{\alpha}{2} \|\mathbf{W}\|_2^2 = \frac{\alpha}{2} \sum_i \sum_j w_{ij}^2 \qquad \ell_1 : \Omega(\mathbf{W}) = \alpha \|\mathbf{W}\|_1 = \alpha \sum_i \sum_j |w_{ij}|$$

Beispiel: binary cross entropy mit ℓ_2 -Regularisierung:

$$J_{\text{reg}}(\boldsymbol{\omega}, \mathbf{X}) = J(\boldsymbol{\omega}, \mathbf{X}) + \Omega(\boldsymbol{\omega}) = -\frac{1}{N} \sum_{i=0}^N (y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})) + \frac{\alpha}{2} \frac{1}{N} \sum_l \sum_i \sum_j (\omega_{ij}^{[l]})^2$$

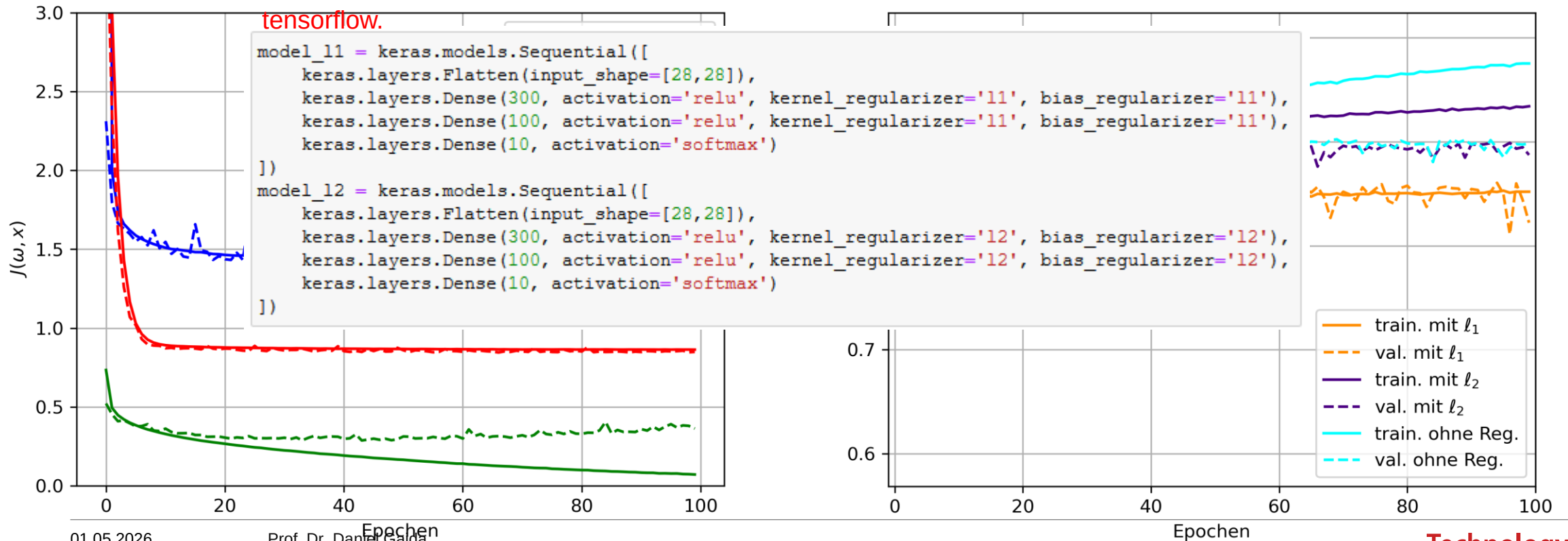
Künstliche Neuronale Netze - KNN

Beispiel Bildklassifizierer: Regularisierung

Vergleich zweier Modelle, eines mit Regularisierung eines ohne

Verlustfunktion

Metrik



01.05.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Seite 73

Quelle: Patrick S., Hahn-Schickard

Künstliche Neuronale Netze - KNN

Regularisierung

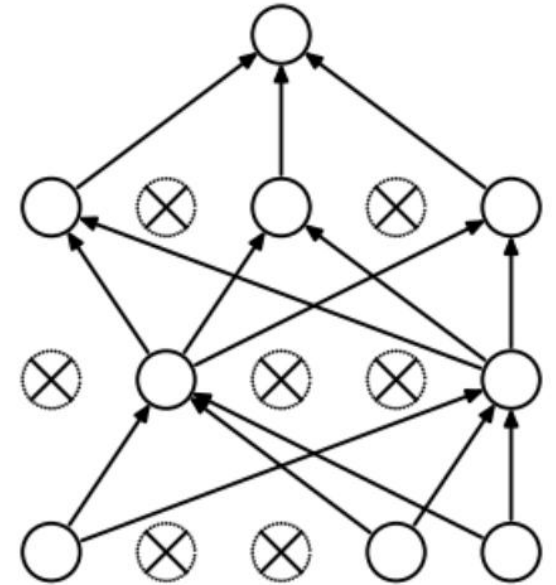
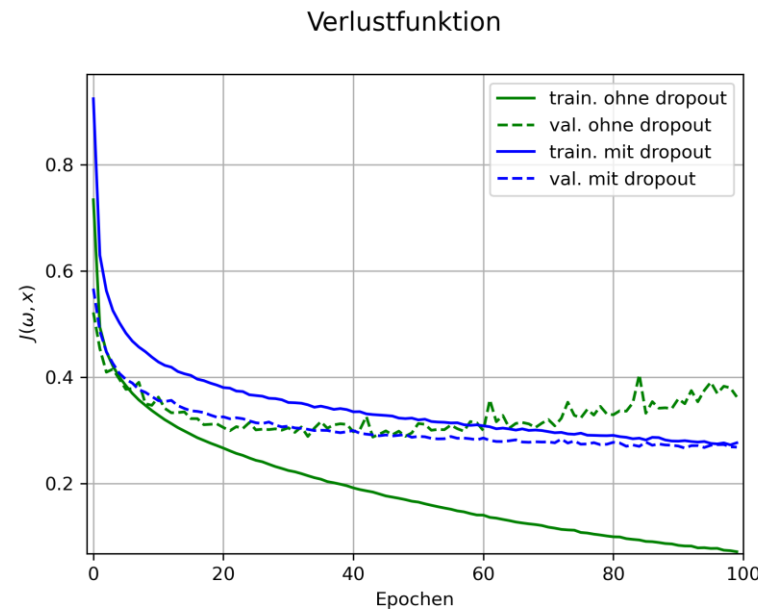
Problemstellung:

Overfitting auf Trainingsdaten $\longrightarrow$ **Regularisierung**

Dropout: Zufälliges deaktivieren von Neuronen während des Trainings. Reduziert Komplexität des neuronalen Netzes.

```
from tensorflow.keras.layers import Dropout
```

```
model = Sequential()
model.add(Flatten(input_shape=[28, 28]))
model.add(Dense(units=300, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(units=100, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(units=10, activation='softmax'))
```



Künstliche Neuronale Netze - KNN

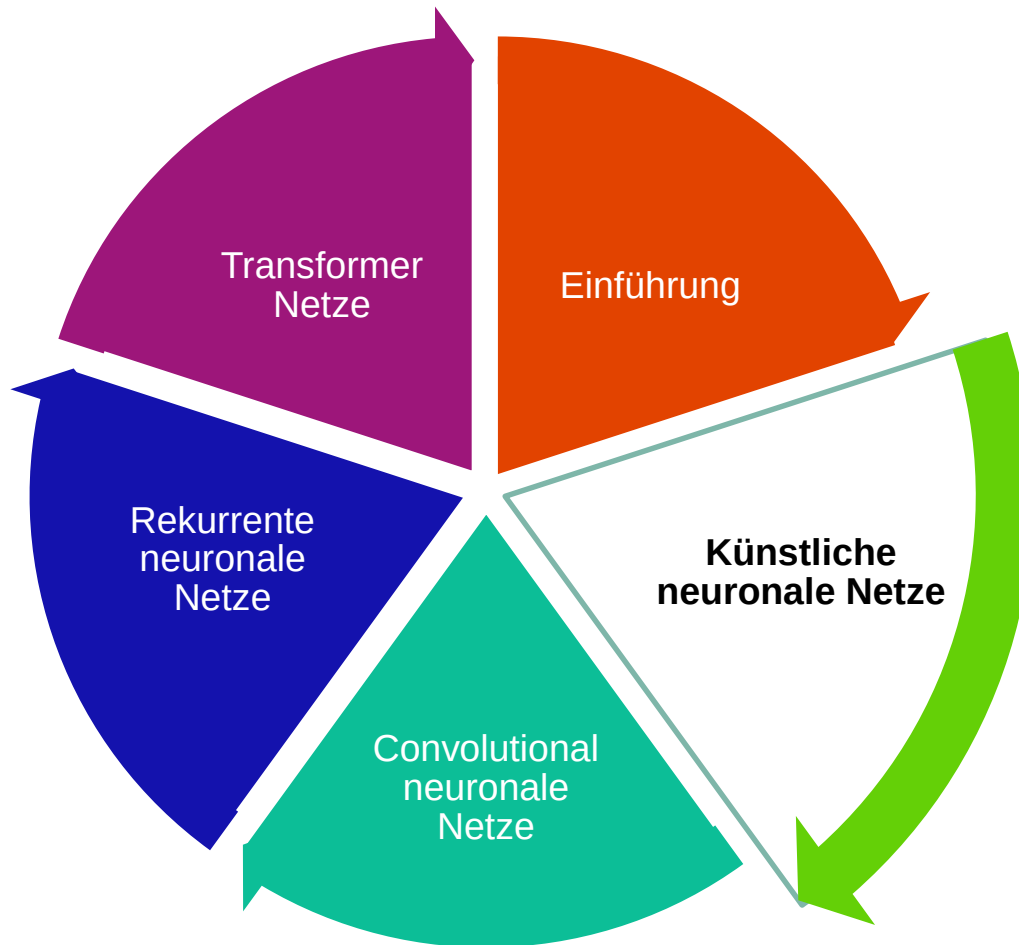
Regularisierung: Übung

Experimentieren Sie mit verschiedenen Regularisierungsmethoden:

- Early Stopping
- l1, l2 Regularisierung - [05_fashion_mnist_10_l1_l2_regularizers.ipynb](#)
- Dropout - [06_fashion_mnist_10_dropout.ipynb](#)
- Integrieren Sie Early Stopping in die beiden anderen Skripte.
 - Experimentieren Sie mit den Argumenten min_delta und patience
- Wie unterscheiden sich die Ergebnisse?

Deep Learning

Ausblick



Künstliche neuronale Netze: Part 2

- Trainings-, Test- und Validierungsdaten
- Regularisierung
 - Early Stopping
 - ℓ_1 & ℓ_2 -Regularisierung
 - Dropout
 - **Gewichtsinitialisierung**
 - **Batch Normalisierung**
- Optimierungsalgorithmen

Deep Learning - Zutaten

Modellarchitektur

- Anzahl Schichten
- Aktivierungsfunktion

Kostenfunktion

Definiert was ein gutes neuronales Netz ist

Optimierungsalgorithmus

Minimiert die Kostenfunktion, indem es Gewichte des NNs variiert

Training des NNs

Minimierung der Kostenfunktion

Künstliche Neuronale Netze - KNN

Optimierungsalgorithmen I

- Vanilla SGD Gradientenabstieg
 - SGD mit Momentum
 - Berücksichtigt vorherige Gradienten (Momentvektor), $0 < \beta < 1$
 - Schnellere Konvergenz und weniger Oszillation
 - Nesterov Accelerated Gradient (NAG)
 - Messung des Gradienten an $\omega + \beta \mathbf{m}$ statt ω
 - $\mathbf{m}$ zeigt gewöhnlich in Richtung Optimum
 - Fast immer schneller als Vanilla Momentum Optimierung
 - Reduziert Oszillationen
 - RMSProp
 - Ersetzt Gradientenakkumulation durch exponentiell gewichteten gleitenden Mittelwert
- $\omega \leftarrow \omega - \eta \nabla_{\omega} J(\omega)$
 - $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\omega} J(\omega)$
 - $\omega \leftarrow \omega + \mathbf{m}$
 - $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\omega} J(\omega + \beta \mathbf{m})$
 - $\omega \leftarrow \omega + \mathbf{m}$
 - $\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\omega} J(\omega) \otimes \nabla_{\omega} J(\omega)$
 - $\omega \leftarrow \omega - \eta \nabla_{\omega} J(\omega) \oslash \sqrt{\mathbf{s} + \epsilon}$

Künstliche Neuronale Netze - KNN

Optimierungsalgorithmen I

- Vanilla SGD Gradientenabstieg
- RMSProp
 - Ersetzt Gradientenakkumulation durch exponentiell gewichteten gleitenden Mittel

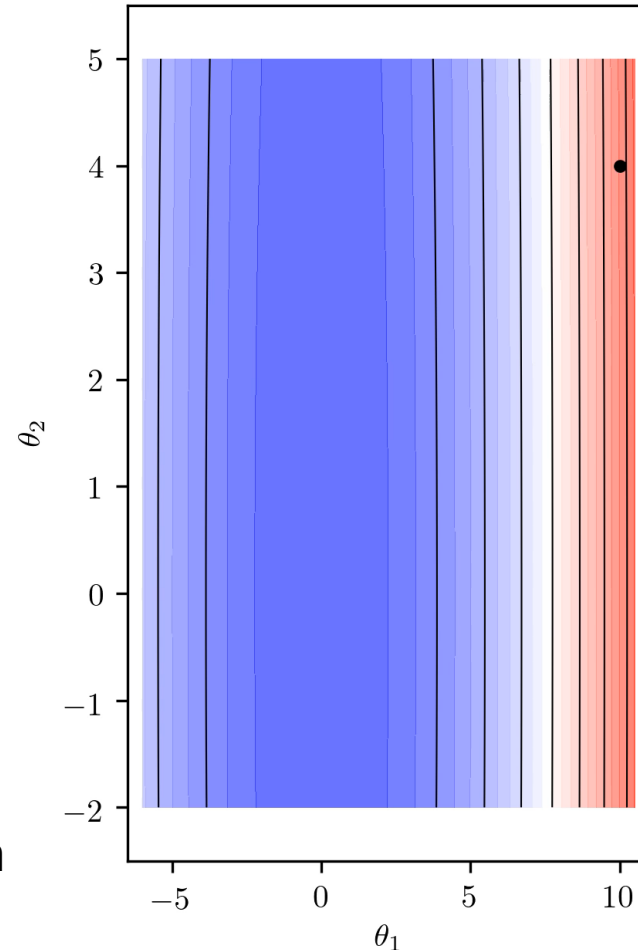
$$\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\omega} J(\omega) \otimes \nabla_{\omega} J(\omega)$$

$$\omega \leftarrow \omega - \eta \nabla_{\omega} J(\omega) \oslash \sqrt{\mathbf{s} + \epsilon}$$

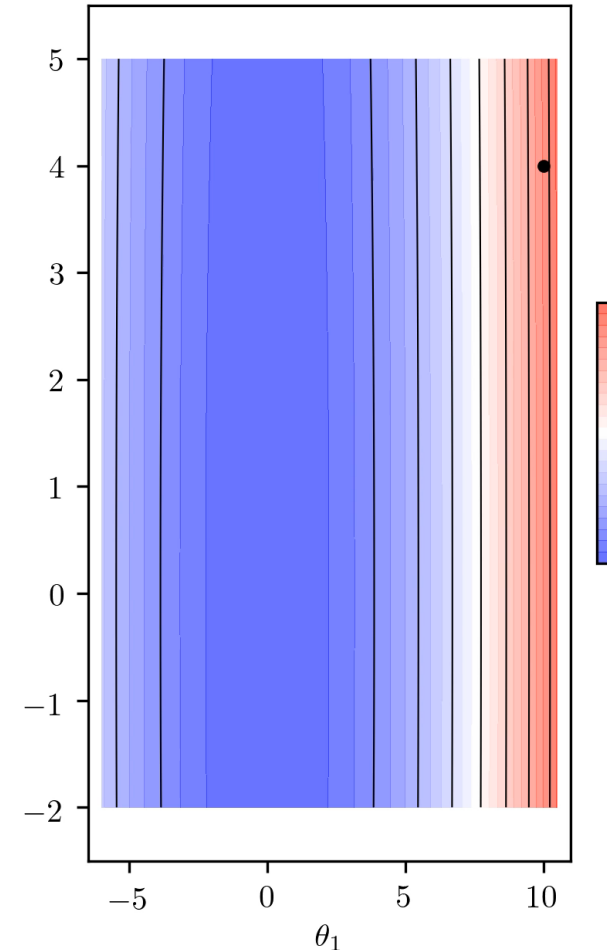


Komponentenweise Multiplikation und Division

Alg.: RMSProp, η : 0.40, β_1 : 0.9
epoch: 0, $J(\theta)$: 100.6, $\Delta\theta_k$: 1.79



Alg.: GD, η : 0.40,
epoch: 0, $J(\theta)$: 100.6, $\Delta\theta_k$: 8.00



Künstliche Neuronale Netze - KNN

Optimierungsalgorithmen II

1 Adam Algorithmus

Der Adam Algorithmus wird in Algorithmus 1 dargestellt. Die Berechnung des 1. und des 2. Moments sind aus dem Momentum Optimierung Algorithmus bzw. des RMSProp Algorithmus bekannt.

Algorithm 1 Adam Algorithmus

Require: $\eta > 0$: Schrittweite

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponentielle Abbauraten für 1. und 2. Moment

Require: $J(\omega)$: Zielfunktion

Require: ω_0 : Startvektor

$\mathbf{m}_0 \leftarrow 0$ (1. Moment auf Null initialisieren)

$\mathbf{v}_0 \leftarrow 0$ (2. Moment auf Null initialisieren)

$t \leftarrow 0$ (Zeitschritt auf Null initialisieren)

while ω_t not converged **do**

$t \leftarrow t + 1$

▷ Aktualisiere Zeitschritt

$\mathbf{g}_t \leftarrow \nabla_{\omega} J(\omega_{t-1})$

▷ Erhalte Gradienten in Schritt t

$\mathbf{m}_t \leftarrow \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t$

▷ Aktualisiere 1. Moment

$\mathbf{v}_t \leftarrow \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t \odot \mathbf{g}_t$

▷ Aktualisiere 2. Moment

$\widehat{\mathbf{m}}_t \leftarrow \mathbf{m}_t / (1 - \beta_1^t)$

▷ Bias im 1. Moment korrigieren

$\widehat{\mathbf{v}}_t \leftarrow \mathbf{v}_t / (1 - \beta_2^t)$

▷ Bias im 2. Moment korrigieren

$\omega_t \leftarrow \omega_{t-1} - \eta \cdot \widehat{\mathbf{m}}_t \odot (\sqrt{\widehat{\mathbf{v}}_t} + \epsilon)$

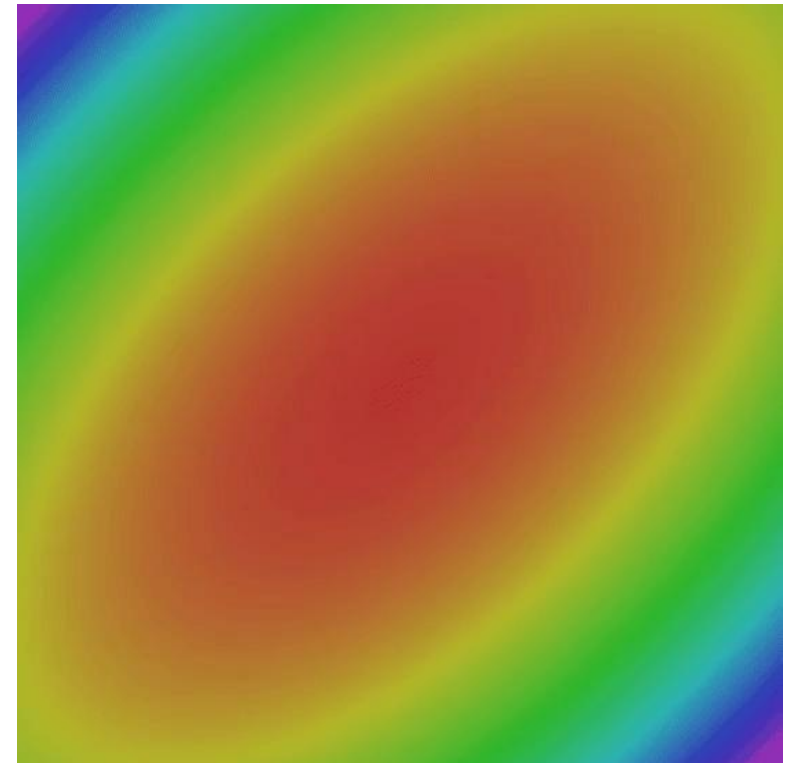
▷ Aktualisiere die Parameter

end while

return ω_t

▷ Rückgabe der Endparameter

— SGD — RMSProp
— SGD+Momentum — Adam



Künstliche Neuronale Netze - KNN

Optimierungsalgorithmen in Keras

```
# Adadelta
...
optimizer="Adadelta"
optimizer=keras.optimizers.Adadelta()
...

# Adagrad
...
optimizer="Adagrad"
optimizer=keras.optimizers.Adagrad()
...

# Adam
...
optimizer="Adam"
optimizer=keras.optimizers.Adam()
...
```

```
# Nadam
...
optimizer="Nadam"
optimizer=keras.optimizers.Nadam()
...

# RMSprop
...
optimizer="RMSprop"
optimizer=keras.optimizers.RMSprop()
...

# SGD
...
optimizer="SGD"
optimizer=keras.optimizers.SGD()
...
```

Künstliche Neuronale Netze - KNN

Trainingsverfahren Zusammenfassung

Trainingsverfahren

1. Gewichte initialisieren (zufällig oder nach einem Schema)
2. Propagiere Trainingsdaten vorwärts durch das Netz
3. Berechne den Fehler zwischen der Zielausgabe und der tatsächlichen Ausgabe
4. Propagiere den Fehler rückwärts und berechne den anteiligen Fehler aller Neuronen
5. Aktualisiere die Gewichte nach einem Optimierungsverfahren (Gradientenabstieg)
6. Ist der Validierungsfehler größer als ein gewünschter Betrag, springe zu Punkt 2.
Ansonsten beende das Training

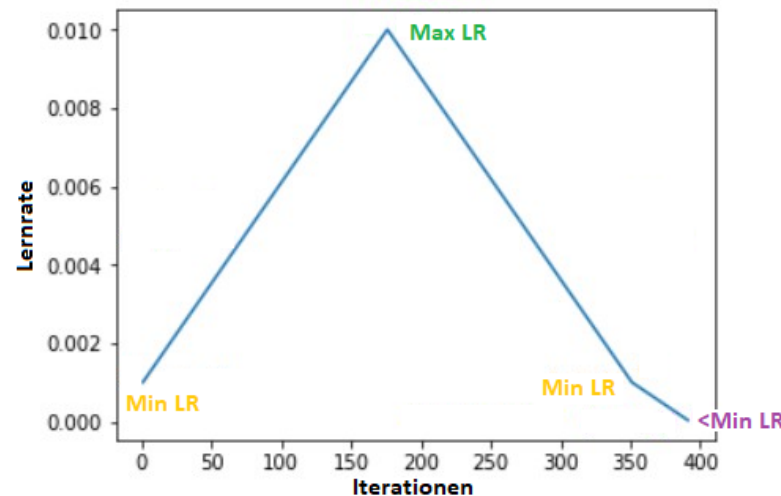
Künstliche Neuronale Netze - KNN

Praxisleitfaden

Hyperparameter	Standardwert
Gewichtsinitialisierung	HE Normal
Aktivierungsfunktion	ELU
Normalisierung	Batchnormalisierung bei Deep
Regularisierung	Early Stopping ($+\ell_2$ falls nötig)
Optimierer	SGD mit Momentum (oder RMSProp oder Nadam)
Lernrate	Ein-Zyklus-Politik (1Cycle policy)*

Ein-Zyklus-Politik

- Maximale Lernrate: LR range test
- Minimale Lernrate: $\frac{1}{5}$ oder $\frac{1}{10}$ der maximalen Lernrate
- Zykluslänge: $<$ Totalanzahl an Epochen
- Letzte Iterationen: $LR < \frac{1}{10}$ oder $\frac{1}{100}$



Künstliche Neuronale Netze - KNN

Fragen III

Welche Aussagen über lineare Aktivierungsfunktionen sind wahr?

- Nur binäre Ausgaben sind möglich
- Ermöglicht nicht-lineare Zuordnung zwischen Ein- und Ausgängen
- Gradientenabstieg ist nicht möglich, da die Ableitung eine Konstante ist

Künstliche Neuronale Netze - KNN

Bildklassifizierer mit Keras

MNIST Fashion Datensatz

Vollständiges Beispiel:

[07_fashion_mnist_classification.ipynb](#)



Nächste Vorlesung und Fragen

- Convolutional Neuronale Netze
- Fragen?

Künstliche Neuronale Netze - KNN

Notationen

- N : Anzahl an (Trainings-)Instanzen
- p : Dimension der Eingabe (Anzahl der Features)
- K : Dimension der Ausgabe (oder Anzahl der Klassen)
- L : Anzahl der Schichten eines neuronalen Netzes
- $\mathbf{x}^{(i)} \in \mathbb{R}^p / \mathbf{y}^{(i)} \in \mathbb{R}^K$: der i -te Eingabevektor / Ausgabevektor dargestellt als Zeilenvektor
- $x_j^{(i)} / y_k^{(i)}$: Das j -te Element des i -te Eingabevektors / Ausgabevektor (Skalar)
- $\mathbf{X} \in \mathbb{R}^{N \times p} / \mathbf{Y} \in \mathbb{R}^{N \times K} : \mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^T \\ \vdots \\ (\mathbf{x}^{(N)})^T \end{pmatrix} = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \\ \vdots & \vdots & & \vdots \\ x_1^{(N)} & x_2^{(N)} & \dots & x_p^{(N)} \end{pmatrix}$ Eingabematrix
- $\omega_{ji}^{[\ell]}$: Gewicht des j -ten Eingangs des i -ten Neurons der ℓ -ten Schicht (alternativ θ)
- $\mathbf{W}^{[\ell]} \in \mathbb{R}^{(\text{Anzahl Neuronen der Schicht } \ell) \times (\text{Anzahl Neuronen der Schicht } \ell-1)}$: Gewichtsmatrix der Schicht ℓ .
- $\hat{\mathbf{y}}^{(i)} \in \mathbb{R}^K, \hat{\mathbf{y}}^{(i)} = h(\mathbf{x}^{(i)})$: Ist der vorhergesagte Ausgabevektor (Estimator)
 h : ist die als Hypothese bezeichnete Vorhersagefunktion Ihres Systems
- $\sigma^{[\ell]}(\cdot)$: Aktivierungsfunktion der ℓ -ten Schicht, für die Stufenfunktion schreiben wir $\sigma(\cdot) = f(\cdot)$